

PROPOSAL SKRIPSI

**PENGEMBANGAN MODEL FEDERATED LEARNING
DENGAN AUTOENCODER UNTUK DETEKSI ANOMALI
DATA SENSOR IOT PERTANIAN TOMAT**



MUHAMMAD RIZKI

F1E122140

**PROGRAM STUDI SISTEM INFORMASI
JURUSAN TEKNIK ELEKTRO DAN INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS JAMBI**

PROPOSAL SKRIPSI

**PENGEMBANGAN MODEL FEDERATED LEARNING
DENGAN AUTOENCODER UNTUK DETEKSI ANOMALI
DATA SENSOR IOT PERTANIAN TOMAT**

Diajukan sebagai salah satu syarat untuk memperoleh
Gelar Sarjana pada Program Studi Sistem Informasi



MUHAMMAD RIZKI

F1E122140

**PROGRAM STUDI SISTEM INFORMASI
JURUSAN TEKNIK ELEKTRO DAN INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS JAMBI**

HALAMAN PENGESAHAN

PROPOSAL SKRIPSI

**” PENGEMBANGAN MODEL FEDERATED LEARNING
DENGAN AUTOENCODER UNTUK DETEKSI ANOMALI
DATA SENSOR IOT PERTANIAN TOMAT ”**

Oleh:

MUHAMMAD RIZKI

F1E122140

Disetujui:

Pembimbing Utama

Pembimbing Pendamping

Muh. Subhan, S.ST., M.T.
NIP. 198608122014041001

Akhiyar Waladi, S.Komp., M.Kom.
NIP. 199612162022032016

Diketahui:

Dekan Fakultas Sains
dan Teknologi

Ketua Jurusan Teknik Elektro
dan Informatika

Drs. Jefri Marzal, M.Sc., D.I.T.
NIP. 196806021993031004

Edi Saputra, S.T., M.Sc.
NIP.198501082015041003

DAFTAR ISI

HALAMAN PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR TABEL	iv
DAFTAR GAMBAR	v
I. PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah.....	5
1.3 Tujuan Penelitian	5
1.4 Batasan Masalah	5
1.5 Manfaat Penelitian	6
II. TINJAUAN PUSTAKA	7
2.1 Internet of Things (IoT)	7
2.2 Smart Farming	9
2.3 Deteksi Anomaly.....	10
2.4 Autoencoder	12
2.5 Federated Learning	14
2.6 Penelitian Terdahulu.....	16
III. METODOLOGI PENELITIAN	20
3.1 Waktu dan Tempat Penelitian	20
3.2 Bahan dan Alat Penelitian.....	20
3.3 Jenis dan Sumber Data	20
3.4 Tahapan Penelitian	21
3.5 Tahapan Pendahuluan.....	23
3.6 Preprocessing Data	25
3.7 Data Training	32
3.8 Threshold Anomali.....	38
3.9 Deteksi Anomali	43

3.10	Evaluasi Model Federated Autoencoder	48
DAFTAR PUSTAKA.....		52

DAFTAR TABEL

Tabel 1. Tahapan Federated Learning	15
Tabel 2. Penelitian terdahulu.....	17
Tabel 3. Perbandingan penelitian.....	23
Tabel 4. Total data setelah pemisahan	27
Tabel 5. Model training autoencoder	35

DAFTAR GAMBAR

Gambar 1. Arsitektur Autoencoder	13
Gambar 2. Alur proses Federated Learning	15
Gambar 3. Dataset Penelitian	21
Gambar 4. Tahapan Penelitian	22
Gambar 5. Tahapan pre-processing data	25
Gambar 6. Dataset sebelum pemisahan.....	26
Gambar 7. Data sebelum cleaning	28
Gambar 8. Data setelah cleaning	28
Gambar 9. Alur data training	32
Gambar 10. Proses deteksi anomali	44

I. PENDAHULUAN

1.1 Latar Belakang

Pertanian modern mengalami transformasi signifikan melalui penerapan teknologi digital berbasis Internet of Things (IoT) dan Artificial Intelligence (AI) yang mendorong peningkatan produktivitas dan efisiensi operasional. Pemanfaatan AI dalam pengelolaan pertanian, seperti rekomendasi irigasi, pemupukan, dan pengendalian hama berbasis data sensor, dilaporkan mampu meningkatkan hasil panen rata-rata hingga 25% dibandingkan metode konvensional (Aijaz et al., 2025). Transformasi ini mencerminkan pergeseran paradigma dari praktik pertanian berbasis pengalaman menuju pengambilan keputusan berbasis data dan analitik prediktif yang lebih terukur.

Perkembangan tersebut berlangsung seiring dengan pertumbuhan pesat perangkat IoT secara global. Jumlah perangkat IoT diproyeksikan terus meningkat dan diperkirakan dapat mencapai hingga 40 miliar unit pada tahun 2030, yang menyebabkan lonjakan signifikan pada volume dan kompleksitas data yang dihasilkan. Skala pertumbuhan ini tidak hanya membuka peluang pemanfaatan data secara luas, tetapi juga menghadirkan tantangan teknis yang serius, terutama terkait keandalan sistem, kualitas data, dan pengelolaan data dalam lingkungan yang terdistribusi (Krzyśztoń et al., 2024). Penelitian tersebut menegaskan bahwa sistem IoT berskala besar sangat rentan terhadap gangguan data, sehingga memerlukan pendekatan analitik yang mampu menangani data tidak stabil dan bersifat heterogen.

Dalam konteks pertanian, integrasi IoT dan AI terbukti meningkatkan efisiensi penggunaan sumber daya. Sistem irigasi presisi berbasis sensor IoT dilaporkan mampu mengurangi konsumsi air hingga 20% melalui pemantauan kondisi lahan secara real-time dan otomatisasi proses penyiraman (Aijaz et al., 2025). Namun, efektivitas sistem ini sangat bergantung pada kualitas dan keandalan data sensor yang digunakan. Ketika data sensor tidak akurat atau tidak stabil, keputusan yang dihasilkan sistem cerdas berpotensi menjadi tidak optimal, sehingga justru dapat menurunkan kinerja sistem pertanian secara keseluruhan..

Peningkatan jumlah perangkat IoT di sektor pertanian menghasilkan data sensor dalam jumlah besar dan menciptakan peluang penerapan algoritma *machine learning* (ML) untuk analisis pola data secara lebih cerdas. ML dan AI

tidak hanya digunakan untuk memprediksi hasil panen, tetapi juga dalam deteksi penyakit tanaman, hingga pemantauan kondisi lahan secara adaptif (Mohammed, 2025). Pemanfaatan data sensor tersebut memberikan manfaat besar, namun kualitas data sensor pertanian sering kali tidak stabil. Ketidakstabilan ini memicu munculnya anomali data, yaitu nilai-nilai yang menyimpang dari pola normal. (Adkisson et al., 2022) menjelaskan bahwa anomali dapat muncul akibat gangguan komunikasi, kalibrasi yang keliru, hingga kegagalan sensor, dan jika tidak segera dideteksi dapat mengakibatkan keputusan operasional yang tidak tepat, seperti kesalahan pengaturan irigasi atau pemantauan lahan.

Keandalan data sensor memegang peranan penting salah satu contohnya dalam budidaya tomat, mengingat komoditas ini memiliki nilai ekonomi tinggi dan memerlukan manajemen irigasi yang sangat presisi untuk menghasilkan panen yang optimal (Martelli et al., 2025). Sistem irigasi berbasis sensor IoT memungkinkan penyiraman air dilakukan secara otomatis berdasarkan kebutuhan tanaman. Namun, ketika data sensor yang digunakan sebagai dasar pengambilan keputusan mengandung anomali misalnya akibat kerusakan sensor atau gangguan lingkungan maka keputusan irigasi dapat menjadi tidak akurat. Kondisi tersebut berpotensi menurunkan produktivitas, meningkatkan risiko stres tanaman, dan bahkan menyebabkan kegagalan panen.

Penelitian terdahulu telah menerapkan metode deteksi anomali berbasis *deep learning*, termasuk penggunaan Autoencoder. Model ini dapat mempelajari pola deret waktu secara mendalam dan mengidentifikasi data yang menyimpang dari pola normal. Lachehab et al., (2024) menunjukkan bahwa Autoencoder memberikan *reconstruction error* yang lebih akurat untuk deteksi anomali pada getaran mesin. Lee et al., (2024) menemukan bahwa Autoencoder efektif dalam mendeteksi anomali penggunaan energi pada sistem *smart metering* secara real-time.

Sebagian besar penelitian yang ada masih menggunakan skema pelatihan terpusat (*centralized learning*), di mana seluruh data harus dikirim ke satu server untuk membangun model secara keseluruhan. Pendekatan ini kurang sesuai untuk sistem irigasi IoT yang terdistribusi, terutama karena risiko kebocoran privasi data petani, biaya komunikasi yang tinggi, dan keterbatasan jaringan yang menghambat pengiriman data dalam jumlah besar (Dembani et al., 2025). Kondisi tersebut menunjukkan perlunya pendekatan alternatif yang mampu melakukan pelatihan model secara terdistribusi tanpa memindahkan data mentah, sekaligus menjaga efisiensi komunikasi dan privasi data.

Salah satu pendekatan yang ditawarkan untuk mengatasi masalah ini adalah Federated Learning (FL). Federated Learning memungkinkan model dilatih secara terdistribusi pada perangkat lokal misalnya pada setiap lahan pertanian atau node sensor tanpa harus mengirimkan data mentah ke server pusat. Hanya parameter model atau *gradient* dari hasil pelatihan lokal yang dikirim untuk digabungkan menjadi model global. Dengan mekanisme tersebut, FL mampu mengurangi beban jaringan, menjaga privasi data, dan meningkatkan efisiensi kolaborasi antar lokasi (Mohammed, 2025)

Beberapa penelitian terbaru telah menyoroti potensi penerapan Federated Learning (FL) dalam sektor pertanian. Studi oleh Mohammed (2025) menjelaskan bahwa FL dapat dimanfaatkan dalam sistem pertanian cerdas untuk meningkatkan efisiensi pengolahan data, prediksi hasil panen, serta deteksi penyakit tanaman secara kolaboratif tanpa perlu menggabungkan data mentah dari setiap lahan, sementara Dembani et al., (2025) menegaskan bahwa FL membuka peluang kolaborasi lintas petani tanpa pelanggaran privasi data. Namun, sebagian besar penelitian tersebut masih berfokus pada klasifikasi atau prediksi hasil panen, bukan pada deteksi anomali deret waktu data sensor pertanian.

Penelitian di bidang Industrial IoT (IIoT) menunjukkan bahwa penggabungan autoencoder dengan federated learning mampu memberikan hasil yang efektif untuk deteksi anomali pada sistem industri dan jaringan. Becker et al., (2022) mengusulkan model federated autoencoder untuk *condition monitoring*, dan hasilnya menunjukkan peningkatan akurasi deteksi anomali tanpa perlu mengumpulkan data ke satu pusat. (Lim et al., 2025) juga mengonfirmasi bahwa penerapan federated learning (FL) umumnya masih berfokus pada deteksi anomali di jaringan komputer dan perangkat IoT, sedangkan pemanfaatannya pada data sensor IoT di pertanian yang bersifat beragam dan berukuran besar masih belum banyak dikembangkan secara maksimal.

Kondisi tersebut menunjukkan kesenjangan penelitian yang signifikan antara penerapan FL pada domain industri dan pada domain pertanian berbasis IoT. Jika di sektor industri FL-Autoencoder telah berhasil meningkatkan akurasi dan efisiensi deteksi anomali, maka di sektor pertanian penerapannya masih memerlukan adaptasi terhadap karakteristik data sensor lingkungan yang dinamis, tidak seragam, dan terdistribusi di berbagai lokasi lahan (Dembani et al., 2025). Oleh karena itu, penelitian ini berupaya mengisi kesenjangan tersebut dengan mengembangkan model Federated Learning berbasis Autoencoder untuk deteksi anomali pada data sensor pertanian terdistribusi. Dalam model ini, proses

pelatihan dilakukan secara lokal pada masing-masing klien yang merepresentasikan lahan atau node sensor dan hasil parameter pelatihan dikirim ke server pusat untuk diagregasikan menjadi model global tanpa memindahkan data mentah. Pendekatan ini diharapkan tidak hanya meningkatkan akurasi deteksi anomali, tetapi juga menjaga privasi data petani serta efisiensi komunikasi antar klien melalui proses pelatihan terdistribusi yang aman dan adaptif.

Penelitian ini menggunakan metode eksperimental kuantitatif dengan memanfaatkan data sensor pertanian yang diperoleh dari publikasi ilmiah sebelumnya. Dataset tersebut memuat informasi kondisi lingkungan pada budidaya tomat dengan berbagai skema irigasi, mencakup parameter seperti kelembaban tanah, suhu, serta karakteristik ruang terbuka yang umum ditemukan pada sistem pertanian berbasis IoT (Belli et al., 2025). Penggunaan data nyata ini memungkinkan pengujian model dilakukan pada kondisi yang lebih representatif terhadap dinamika lingkungan lapangan, sehingga evaluasi kinerja Federated Learning berbasis Autoencoder dalam mendeteksi anomali dapat dilakukan secara lebih akurat pada data sensor. Pendekatan ini juga dirancang untuk mencapai keseimbangan antara akurasi deteksi, efisiensi komunikasi, dan perlindungan privasi data antar klien yang merepresentasikan lokasi lahan pertanian berbeda.

Penelitian ini berkontribusi dengan memperluas penerapan Federated Learning (FL) pada sistem pertanian berbasis IoT. Model yang dikembangkan dalam penelitian ini menggabungkan Federated Learning dengan Autoencoder untuk melakukan deteksi anomali pada data sensor IoT Tomat. Melalui federated learning, model ini diharapkan mampu beradaptasi dengan karakteristik data sensor irigasi tomat yang bersifat tidak terpusat (non-IID), dinamis, dan heterogen di tiap lokasi lahan. Pendekatan ini juga berpotensi mempertahankan akurasi deteksi anomali meskipun terdapat variasi kondisi lingkungan dan perbedaan distribusi data antar klien (Laridi et al., 2024)

Secara praktis penelitian ini diharapkan mampu menghadirkan solusi yang dapat diterapkan pada berbagai skala pertanian, baik untuk petani kecil maupun besar, guna meningkatkan keandalan dan ketahanan sistem pemantauan lahan pertanian. Pendekatan Federated Learning memungkinkan pelatihan model dilakukan tanpa harus mengirimkan data mentah, sehingga menjaga kerahasiaan informasi sensor di tiap lokasi (R. Vidhya et al., 2025). Dengan demikian, penelitian berjudul *“Pengembangan Model Federated Learning dengan Autoencoder untuk Deteksi Anomali Data Sensor IoT Pertanian Tomat”*

diharapkan dapat menjadi inovasi strategis dalam mendukung pertanian cerdas (smart farming) yang efisien, aman, dan berkelanjutan, sekaligus membuka peluang penelitian lanjutan di bidang analitik data pertanian terdistribusi berbasis kecerdasan buatan.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana mengembangkan model Federated Learning dengan Autoencoder yang mampu melakukan pelatihan secara terdistribusi?
2. Bagaimana kinerja model Federated Learning berbasis Autoencoder dalam mendeteksi data anomali ?

1.3 Tujuan Penelitian

Tujuan penelitian ini disusun untuk menjawab rumusan masalah di atas, yaitu:

1. Mengembangkan model Federated Learning dengan Autoencoder yang mampu melakukan proses pelatihan secara terdistribusi tanpa memindahkan data mentah dari setiap node sensor.
2. Mengevaluasi kinerja model Federated Learning berbasis Autoencoder dalam mendeteksi anomali pada data sensor IoT.

1.4 Batasan Masalah

Agar penelitian ini lebih fokus dan tidak melebar dari tujuan utama, maka ruang lingkup penelitian dibatasi pada hal-hal berikut:

1. Data yang digunakan merupakan data sensor IoT pertanian tomat yang berasal dari publikasi ilmiah sebelumnya (Belli et al., 2024), bukan data yang dikumpulkan secara langsung dari perangkat IoT di lapangan.
2. Data yang digunakan berupa data sensor tanah (soil sensor) dari dataset *stuard_soil_data* yang dihasilkan oleh perangkat Milesight EM500 SMTTC.
3. Metode deteksi anomali yang digunakan adalah Autoencoder dengan pendekatan unsupervised learning
4. Model Federated Learning pada penelitian ini dibatasi pada penggunaan 3 klien, 1 federated rounds, dan metode agregasi FedAvg
5. Penelitian ini hanya mengevaluasi kinerja model berdasarkan nilai reconstruction error, dan tingkat akurasi dalam mendeteksi anomali, tanpa membahas penerapannya langsung pada perangkat IoT fisik.

1.5 Manfaat Penelitian

Hasil penelitian ini diharapkan bermanfaat untuk:

1. Memberikan model deteksi anomali yang dapat membantu meningkatkan ketepatan analisis data sensor IoT pada sistem irigasi tomat.
2. Menyediakan pendekatan pelatihan model yang lebih aman karena data tidak perlu dibagikan antar lokasi melalui penerapan Federated Learning.
3. Mengurangi kebutuhan transfer data dalam jumlah besar, sehingga penggunaan jaringan menjadi lebih efisien.
4. Menjadi acuan dalam pengembangan sistem pemantauan irigasi berbasis IoT yang lebih stabil dan responsif terhadap perubahan kondisi lingkungan.
5. Memberikan kontribusi bagi penelitian mengenai Federated Learning pada data sensor pertanian yang tersebar di berbagai lokasi.
6. Menjadi referensi bagi pengembangan sistem smart farming berbasis AI pada penelitian selanjutnya.

II. TINJAUAN PUSTAKA

2.1 Internet of Things (IoT)

Internet of Things (IoT) secara umum didefinisikan sebagai konsep yang memungkinkan berbagai perangkat fisik terhubung ke internet sehingga dapat mengumpulkan, mengolah, dan bertukar data secara otomatis tanpa campur tangan manusia. IoT merupakan jaringan objek cerdas yang dibekali sensor dan kemampuan komunikasi sehingga dapat berinteraksi satu sama lain maupun dengan sistem digital. Sementara itu, (Domínguez-Bolaño et al., 2022) menjelaskan IoT sebagai ekosistem perangkat fisik yang terintegrasi dengan teknologi komputasi untuk menyediakan layanan berbasis data secara real-time. Dari perspektif pertanian, (Mansoor et al., 2025) menekankan bahwa IoT adalah teknologi kunci yang memungkinkan pemantauan lingkungan secara terus-menerus untuk mendukung pengambilan keputusan berbasis data.

Secara umum, cara kerja Internet of Things (IoT) mengikuti alur yang bergerak dari perangkat fisik hingga ke aplikasi yang digunakan pengguna. Proses ini dibangun melalui tiga lapisan utama yang bekerja secara berurutan untuk memungkinkan IoT mengumpulkan data, mengirimkan informasi, dan menyediakan layanan digital. Struktur berlapis ini membuat IoT mampu menghubungkan dunia fisik dengan sistem komputasi secara efektif. Berikut adalah penjelasan dari masing-masing lapisan kinerja IoT :

1. Perception Layer

Perception layer berisi berbagai perangkat fisik seperti sensor, aktuator, kamera, RFID, dan modul identifikasi lainnya. Perception layer bertugas mendeteksi kondisi lingkungan dan mengumpulkan informasi seperti suhu, kelembaban, cahaya, atau pergerakan. Selain itu, beberapa perangkat pada lapisan ini juga dapat melakukan tindakan otomatis, misalnya mengaktifkan irigasi, membuka pintu otomatis, atau menyalakan perangkat berdasarkan instruksi sistem. Dengan fungsi tersebut, perception layer menjadi pintu awal yang menghubungkan data dari dunia nyata menuju sistem digital (Domínguez-Bolaño et al., 2022).

2. Network Layer

Network layer adalah bagian yang bertanggung jawab menghubungkan perangkat IoT dengan server atau platform pemrosesan. Lapisan ini memastikan data yang dikumpulkan perception layer dapat

dikirimkan melalui berbagai teknologi komunikasi seperti Wi-Fi, Bluetooth, ZigBee, LoRaWAN, NFC, Ethernet, 4G/5G, hingga jaringan satelit. Pemilihan teknologi ini disesuaikan dengan kebutuhan jarak, kecepatan, kapasitas data, dan konsumsi energi. Selain pengiriman data, network layer juga mengelola keamanan jaringan, enkripsi data, manajemen alamat perangkat, serta pengaturan jalur komunikasi agar data sampai ke tujuan dengan aman dan efisien (Domínguez-Bolaño et al., 2022).

3. Application Layer

Application layer merupakan lapisan yang menyediakan layanan IoT kepada pengguna. Lapisan ini tidak hanya menampilkan data, tetapi juga menjalankan proses analitik, pengendalian jarak jauh, otomasi sistem, manajemen perangkat, dan integrasi dengan teknologi seperti kecerdasan buatan, machine learning, atau cloud computing. Berbagai contoh penerapan lapisan ini dapat ditemukan pada aplikasi rumah pintar, sistem keamanan, pemantauan kesehatan, logistik, hingga dashboard pertanian presisi. Melalui application layer, pengguna dapat berinteraksi secara mudah dengan sistem IoT melalui aplikasi smartphone, web, atau perangkat digital lainnya (Mansoor et al., 2025).

Pemanfaatan Internet of Things (IoT) menjadi salah satu pendorong utama terciptanya *smart farming* atau pertanian presisi. Melalui pemasangan berbagai sensor IoT di area budidaya, kondisi lingkungan seperti kelembaban tanah, suhu udara, serta perubahan iklim dapat dipantau secara terus-menerus. Menurut Mansoor et al., (2025) penggunaan IoT dalam pertanian mampu meningkatkan efisiensi penggunaan air, menekan potensi kerusakan tanaman, dan membantu mendeteksi masalah lebih awal sebelum berdampak pada produktivitas. Dengan akses data yang diperoleh secara real-time, kegiatan budidaya tomat dapat dilakukan dengan lebih akurat dan responsif terhadap kondisi lapangan.

IoT berperan penting karena menyediakan rangkaian data *time series* yang digunakan untuk mendeteksi anomali selama proses budidaya tomat berlangsung. Data yang dikumpulkan dari berbagai sensor di lokasi yang berbeda kemudian dianalisis menggunakan model Federated Learning Autoencoder untuk mengenali pola-pola yang tidak wajar. Selain itu, karakter IoT yang bersifat terdistribusi sangat mendukung penerapan *Federated Learning*, di mana pelatihan model dapat dilakukan langsung pada masing-masing perangkat tanpa perlu memindahkan data mentah ke server pusat. Pendekatan ini tidak hanya

meningkatkan efisiensi komunikasi, tetapi juga memperkuat aspek keamanan dan privasi data.

2.2 Smart Farming

Smart farming atau pertanian cerdas merupakan pendekatan modern dalam pengelolaan pertanian yang memanfaatkan teknologi digital untuk meningkatkan efektivitas, efisiensi, dan ketepatan pengambilan keputusan. Menurut Mansoor et al. (2025), smart farming adalah integrasi teknologi Internet of Things (IoT), sensor pintar, sistem otomatisasi, dan kecerdasan buatan untuk menciptakan proses pertanian yang lebih adaptif dan berbasis data. Pendekatan ini memungkinkan petani memantau kondisi lahan secara real-time, mengoptimalkan penggunaan sumber daya, dan mengurangi risiko kegagalan produksi.

Beberapa ahli memberikan definisi yang memperkuat pemahaman mengenai smart farming. Halawa (2024) menjelaskan bahwa smart farming atau pertanian digital merupakan solusi pintar di sektor pertanian yang memanfaatkan sistem jaringan berbasis Internet of Things (IoT) untuk mendukung pengelolaan yang lebih efisien. Sementara itu, Hasan et al., (2024) menyatakan bahwa smart farming adalah penerapan teknologi cerdas seperti IoT dan deep learning untuk menciptakan pertanian berkelanjutan yang mampu memprediksi kondisi tanaman, mendeteksi penyakit, serta menyesuaikan aktivitas budidaya berdasarkan kebutuhan spesifik tanaman.

Smart farming bekerja dengan mengumpulkan data dari berbagai sumber, misalnya sensor IoT di tanah dan udara, citra udara dari drone, data cuaca, hingga data historis kondisi lahan. Data tersebut digunakan untuk memantau variabel penting seperti kelembaban tanah, suhu udara, intensitas cahaya, tingkat keasaman, dan kondisi iklim mikro. Dengan memanfaatkan data ini, petani dapat menerapkan irigasi presisi, pemupukan terarah, prediksi panen, dan pengendalian hama atau penyakit secara lebih tepat. Mansoor et al. (2025) melaporkan bahwa penggunaan sensor berbasis IoT dalam smart farming dapat meningkatkan efisiensi air hingga 30% dan menurunkan biaya operasional.

Smart farming juga memanfaatkan kecerdasan buatan dan teknik pembelajaran mesin untuk mendeteksi pola, menganalisis tren, dan memprediksi kondisi tanaman. Hasan et al., (2024) menunjukkan bahwa integrasi deep learning dalam smart farming memungkinkan deteksi dini penyakit tanaman melalui analisis data sensor dan citra, sehingga petani dapat melakukan tindakan lebih cepat. Dengan demikian, smart farming tidak hanya berfokus pada

pengumpulan data, tetapi juga pada pengolahan data secara cerdas untuk meningkatkan kualitas dan kuantitas hasil pertanian.

2.3 Deteksi Anomaly

Menurut Kamus Besar Bahasa Indonesia (KBBI), anomali diartikan sebagai kondisi yang tidak normal, menyimpang, atau berbeda dari keadaan umumnya (Sunarto et al., 2022). Dalam konteks data, anomali merujuk pada nilai atau pola yang tidak sesuai dengan perilaku normal yang biasanya muncul dalam suatu himpunan data. Data seperti ini penting untuk diidentifikasi karena dapat memberikan informasi berharga, khususnya dalam pengembangan model analitik yang lebih kuat dan adaptif. Dengan mengenali anomali, model dapat dibangun agar mampu mendeteksi penyimpangan tanpa harus diarahkan pada jenis anomali tertentu.

Dalam perspektif algoritma, konsep anomali sering kali mirip dengan novelty detection, karena keduanya berhubungan dengan kejadian yang tidak selaras dengan pola utama. Perbedaannya, novelty biasanya masih dianggap sebagai bagian dari distribusi data normal apabila model diperbarui, sedangkan anomali tetap diperlakukan sebagai penyimpangan yang perlu diperhatikan secara terpisah (Sivaprasad & Fritz, 2024). Hal ini menunjukkan bahwa pemahaman terhadap struktur distribusi data sangat penting untuk membedakan apakah sebuah nilai termasuk kejadian baru atau benar-benar sebuah anomali.

Deteksi anomali dapat diterapkan pada berbagai jenis data, tidak terbatas hanya pada satu bentuk tertentu. Menurut (Iglesias et al., 2025), teknik deteksi anomali dapat digunakan pada data tabular, citra, maupun data deret waktu (time series) dari sensor IoT. Pada penelitian ini, fokus utama adalah mendeteksi anomali pada data deret waktu yang dihasilkan oleh sensor IoT pertanian. Data seperti ini cenderung dinamis dan dipengaruhi berbagai faktor lingkungan, sehingga keberadaan anomali dapat menandakan gangguan penting seperti kerusakan sensor, kegagalan irigasi, atau kondisi lingkungan yang ekstrem.

Terdapat beberapa jenis algoritma yang dapat digunakan untuk mendeteksi anomali antara lain:

1. K-Means : K-Means merupakan algoritma klusterisasi yang bekerja dengan membagi sekumpulan data ke dalam beberapa klaster berdasarkan tingkat kemiripan antardata. Jumlah klaster ditetapkan terlebih dahulu, kemudian algoritma akan mencari pusat

klaster (*centroid*) dan mengelompokkan setiap titik data pada klaster dengan jarak terdekat. Titik data yang tidak sesuai pola klaster atau berada jauh dari centroid sering dianggap sebagai data yang menyimpang. Oleh karena itu, K-Means dapat dimanfaatkan untuk mendeteksi anomali dengan mengidentifikasi data yang tidak masuk ke dalam kelompok normal (Sunarto et al., 2021).

2. DBSCAN : DBSCAN merupakan algoritma yang dirancang untuk memisahkan kelompok data dengan kepadatan tinggi dari area yang berkepadatan rendah. Namun, algoritma ini sering mengalami kesulitan ketika data memiliki tingkat kepadatan yang bervariasi. Hasil klaster yang dihasilkan juga dapat berbeda-beda bergantung pada titik awal yang dipilih, meskipun jumlah anomali tidak memengaruhi proses pembentukan klaster. Secara umum, tujuan DBSCAN adalah mengelompokkan data normal sekaligus memungkinkan munculnya titik-titik outlier. Namun, beberapa penelitian menyatakan bahwa DBSCAN tidak dioptimalkan secara khusus untuk mendeteksi anomali (Sunarto et al., 2021).

3. Autoencoder : Autoencoder, atau Replicator Neural Networks (RNN), bekerja dengan merepresentasikan data melalui beberapa lapisan tersembunyi dan kemudian berusaha merekonstruksi kembali inputnya. Ketika model telah dilatih dengan baik, ia mampu merekonstruksi data normal dengan akurat. Sebaliknya, data yang bersifat anomali akan menghasilkan nilai kesalahan rekonstruksi yang jauh lebih besar. Karena itu, titik data dengan tingkat kesalahan rekonstruksi tinggi dapat diidentifikasi sebagai anomali (Sunarto et al., 2021).

4. Isolation forest : Isolation Forest (IF) adalah algoritma deteksi anomali yang menawarkan pendekatan berbeda dari metode sebelumnya. Algoritma ini didasarkan pada asumsi bahwa anomali jumlahnya sedikit dan memiliki

karakteristik yang berbeda dari data normal. Alih-alih memodelkan distribusi data dan menghitung jarak antar titik, IF bekerja dengan mengisolasi titik anomali menggunakan struktur pohon yang disebut *Isolation Tree (iTree)*. Melalui penggunaan banyak *iTree* dalam bentuk ensemble dan teknik *subsampling*, algoritma ini mampu menghasilkan model yang ringan, membutuhkan memori minimal, serta memiliki kompleksitas komputasi yang rendah pada tahap pelatihan maupun evaluasi (Liu et al., 2022).

2.4 Autoencoder

Autoencoder merupakan salah satu pendekatan *deep learning* yang digunakan untuk mempelajari representasi tersembunyi (*hidden representation*) dari suatu data melalui proses pembelajaran tanpa label. Secara konsep, autoencoder bekerja dengan memaksa jaringan untuk mempelajari struktur internal data dengan cara merekonstruksi kembali input yang diberikan. Chen & Guo (2023) menjelaskan bahwa autoencoder adalah model jaringan saraf yang membangun peta representasi ke ruang berdimensi lebih rendah agar jaringan dapat menemukan fitur penting di balik data kompleks.

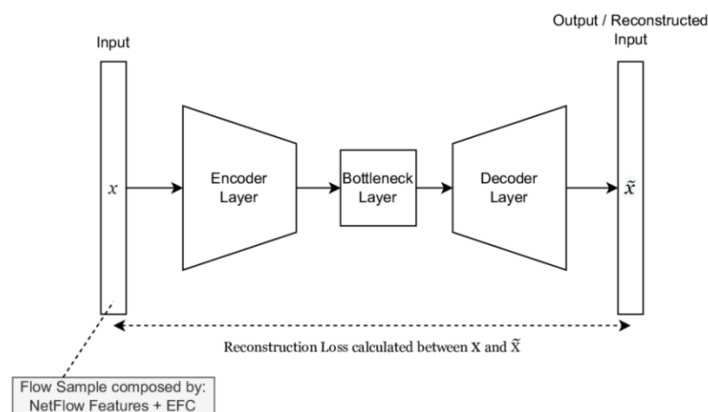
(Walczyna et al., 2025) menekankan bahwa autoencoder merupakan teknik fundamental dalam deteksi anomali karena model belajar menghasilkan rekonstruksi yang berkualitas tinggi untuk pola normal, tetapi gagal melakukannya pada pola yang menyimpang. Hal ini disebabkan oleh kemampuan encoder dalam mengekstraksi fitur dominan dan kemampuan decoder dalam mengungkap keteraturan yang telah dipelajari. Dengan demikian, autoencoder tidak sekadar bekerja sebagai alat kompresi data, tetapi juga sebagai metode untuk memahami pola laten dan hubungan non-linear antarfitur.

Rhachi et al., (2025) menambahkan bahwa autoencoder terutama efektif untuk data berukuran besar dan multidimensi, seperti data jaringan dan sensor IoT. Autoencoder dapat mempelajari relasi antarvariabel yang tidak mudah ditangkap oleh model tradisional, sehingga menghasilkan performa yang sangat kompetitif dalam tugas pendeteksian anomali. Temuan ini menunjukkan bahwa autoencoder memiliki nilai strategis dalam berbagai aplikasi modern, terutama sistem yang membutuhkan kemampuan representasi fitur secara mendalam.

Autoencoder merupakan salah satu arsitektur jaringan saraf tiruan yang digunakan untuk mempelajari representasi tersembunyi dari suatu data melalui pendekatan *unsupervised learning*. Model ini bekerja dengan cara mempelajari pola internal data melalui proses kompresi dan rekonstruksi, di mana jaringan mengubah data input ke dalam bentuk representasi berdimensi lebih rendah, kemudian membangun kembali data tersebut sedekat mungkin dengan bentuk aslinya. Chen & Guo (2023) menjelaskan bahwa proses ini memungkinkan autoencoder mengekstraksi fitur-fitur penting meskipun data memiliki karakteristik non-linear serta berdimensi tinggi.

Secara umum, autoencoder terdiri dari tiga komponen utama, yaitu encoder, latent space, dan decoder. Encoder bertugas mengonversi data input menjadi representasi laten yang lebih ringkas dan mengandung informasi paling relevan dari data. Representasi laten ini kemudian menjadi dasar bagi decoder untuk merekonstruksi kembali data ke bentuk semula. Walczynna et al., (2025) menekankan bahwa mekanisme kompresi dan rekonstruksi ini memaksa jaringan saraf mempelajari pola dominan dalam data, sembari mengurangi komponen yang tidak berkontribusi signifikan terhadap struktur utama.

Arsitektur dasar autoencoder yang menggambarkan alur transformasi input ke latent space dan kembali ke output dapat dilihat pada Gambar berikut.



Gambar 1. Arsitektur Autoencoder

Gambar 1 tersebut menggambarkan arsitektur dasar autoencoder yang terdiri atas tiga komponen utama, yaitu encoder, bottleneck (latent space), dan decoder. Pada bagian kiri, autoencoder menerima masukan berupa vektor data x yang berisi sejumlah fitur sensor. Vektor ini kemudian diproses oleh encoder layer, yaitu rangkaian neuron yang bertugas mengekstraksi pola penting dari data dan mengompresnya ke dalam representasi berdimensi lebih rendah. Tahap ini dikenal sebagai *encoding*, di mana hanya informasi paling relevan yang

dipertahankan. Menurut Chen & Guo (2023), proses encoding tersebut memungkinkan autoencoder mempelajari struktur non-linear dan keterhubungan antar-fitur secara efisien.

Representasi terkompresi tersebut kemudian disimpan pada bottleneck layer, sebagaimana diperlihatkan di bagian tengah gambar. Bottleneck ini berfungsi sebagai ruang representasi laten (*latent representation*) yang menampung informasi inti dari data. Dengan ukuran yang lebih kecil dari input awal, lapisan ini mendorong model untuk melakukan generalisasi dan menghilangkan noise atau informasi yang tidak penting. Walczyna et al., (2025) menyatakan bahwa bottleneck layer merupakan elemen kunci yang memungkinkan autoencoder memahami pola mendasar dalam data normal.

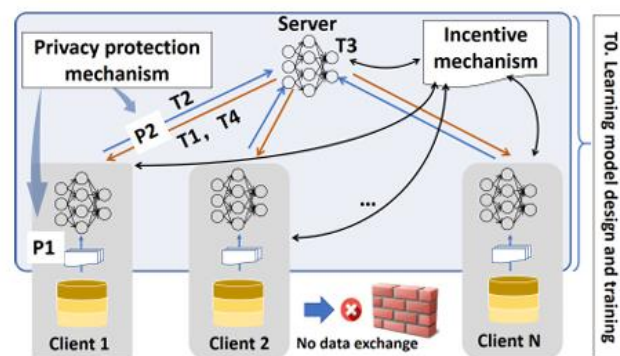
Pada sisi kanan diagram, decoder layer bertugas melakukan proses kebalikan, yaitu *decoding*, untuk merekonstruksi data input ke bentuk semula. Decoder memproyeksikan kembali representasi laten menjadi output rekonstruksi $\hat{x}$, yang berusaha meniru nilai asli input. Jika autoencoder mampu mempelajari pola data dengan baik, hasil rekonstruksi akan sangat mendekati data awal.

Perbedaan antara input dan output dihitung sebagai reconstruction error, yang mengukur seberapa baik model memahami pola data normal. Rekonstruksi yang buruk (error tinggi) menunjukkan bahwa suatu data tidak sesuai dengan pola yang telah dipelajari, sehingga berpotensi dikategorikan sebagai anomali. Temuan Rhachi et al., (2025) menegaskan bahwa autoencoder sangat efektif dalam deteksi anomali pada data IoT, karena data abnormal cenderung sulit direkonstruksi secara akurat.

2.5 Federated Learning

Federated Learning (FL) merupakan pendekatan pembelajaran mesin terdistribusi yang memungkinkan model dilatih secara kolektif tanpa harus memindahkan data mentah ke server pusat. Konsep ini diperkenalkan untuk mengatasi keterbatasan model pembelajaran tradisional yang mengharuskan data dikumpulkan dan dipusatkan, sehingga berpotensi menimbulkan risiko privasi, keamanan, dan penggunaan bandwidth yang besar. Menurut (Brendan McMahan Eider Moore Daniel Ramage Seth Hampson Blaise AgüeraAg & Arcas, 2017) yang pertama kali memperkenalkan Federated Learning, FL dirancang untuk melakukan pembaruan model langsung di perangkat pengguna (on-device training), kemudian mengirimkan parameter model, bukan datanya, untuk digabungkan secara global.

Dalam perkembangannya, sejumlah peneliti memperluas definisi dan ruang lingkup Federated Learning. Y. Liu et al., (2022) menjelaskan bahwa FL merupakan paradigma pembelajaran kolaboratif yang memfasilitasi pelatihan model di berbagai klien heterogen untuk menghasilkan model global yang lebih general. Model ini sangat relevan untuk sistem dengan distribusi data tidak merata (non-IID) serta perangkat dengan kemampuan komputasi yang berbeda-beda. Federated Learning tidak hanya melindungi privasi data, tetapi juga meningkatkan efisiensi komunikasi dan memungkinkan kolaborasi lintas organisasi tanpa melanggar regulasi perlindungan data seperti GDPR.



Gambar 2. Alur proses Federated Learning

Gambar 2 menggambarkan alur utama proses Federated Learning yang melibatkan dua komponen penting, yaitu server pusat dan klien. Pada pendekatan ini, data tidak dikirim ke server, melainkan pemodelan dilakukan secara lokal di perangkat atau node masing-masing klien. Server hanya berfungsi mengoordinasikan pembaruan model, sementara proses pelatihan dilakukan secara terdistribusi di sisi klien (Y. Liu et al., 2022).

Menurut (Y. Liu et al., 2022) Proses Federated Learning berlangsung dalam beberapa tahap berikut :

Tabel 1. Tahapan Federated Learning

Tahap	Nama Proses	Deskripsi
1	Inisialisasi Model oleh Server	Server menyiapkan model awal (arsitektur + parameter awal) serta menentukan algoritma optimasi dan strategi agregasi. Model ini menjadi titik awal federated learning.

Tahap	Nama Proses	Deskripsi
2	Distribusi Model ke Klien	Server mengirim parameter model ke klien terpilih. Pada proses ini tidak ada data yang dikirim, hanya bobot model.
3	Pelatihan Model Lokal di Sisi Klien	Tiap klien melatih model menggunakan data lokal. Proses meliputi forward pass, backward pass, dan pembaruan parameter tanpa membagikan data ke server.
4	Pengiriman Parameter ke Server	Klien mengirimkan hasil pembaruan model (bobot/gradien) ke server. Data mentah tetap berada di sisi klien sehingga privasi terjaga.
5	Agregasi Parameter oleh Server	Server menggabungkan seluruh parameter dari klien menggunakan metode seperti FedAvg, sehingga terbentuk model global yang lebih baik.
6	Distribusi Model Global ke Klien	Model global yang sudah diperbarui dikirim kembali ke klien untuk digunakan dalam putaran pelatihan berikutnya. Proses terus berulang hingga konvergen.

2.6 Penelitian Terdahulu

Terdapat berbagai penelitian terkini yang membahas penerapan Federated Learning (FL) dalam deteksi anomali pada lingkungan data terdistribusi. Pendekatan ini semakin populer karena memungkinkan pelatihan model secara kolaboratif tanpa berbagi data mentah, sehingga menjaga privasi di tengah maraknya sistem IoT dan edge computing. Sejumlah studi menyoroti penggunaan beragam metode seperti autoencoder terdistribusi, graph neural networks, dan ensemble learning, serta berbagai skenario implementasi pada sistem IoT, jaringan komunikasi, dan lingkungan industri. Penelitian-penelitian tersebut tidak hanya mengeksplorasi akurasi deteksi, tetapi juga tantangan seperti komunikasi overhead, heterogenitas data, dan ketahanan terhadap serangan adversarial. Berikut ditampilkan beberapa penelitian terdahulu terkait Federated Learning

Tabel 2. Penelitian terdahulu

No.	Judul Penelitian	Peneliti dan Tahun	Metode	Hasil Penelitian
1.	FedTADBench: Federated Time- series Anomaly Detection Benchmark	Liu et al., 2022	Federated Learning (FedAvg, FedProx, SCAFFOLD, MOON) + 5 algoritma Anomaly Detection (LSTM- AE, USAD, DeepSVDD, GDN, TranAD)	Menyusun benchmark pertama untuk deteksi anomali deret waktu dalam federated learning. Hasil menunjukkan FL dapat menyamai bahkan melampaui model terpusat pada beberapa skenario; FedAvg paling stabil.
2.	Anomaly Detection from Distributed Data Sources via Federated Learning	Cavallin & Mayer, 2022	Federated MLP, Federated GMM, Federated Isolation Forest	Federated Learning mampu menjaga performa mendekati model terpusat terutama pada supervised setting. Kinerja menurun pada data sangat tidak seimbang dan jumlah client yang terlalu banyak.
3.	LSTM- Autoencoder based Anomaly Detection for Indoor Air Quality Time Series Data	Wei et al., 2022	LSTM Autoencoder, Reconstruction Error Thresholding	Model LSTM-AE mampu mendeteksi anomali pada data kualitas udara berbasis CO ₂ dengan akurasi tinggi. Rekonstruksi data normal sangat baik, dan model menunjukkan performa lebih unggul dibandingkan metode pembanding.

4,	Anomaly Detection In IoT Sensor Data Using Machine Learning Techniques For Predictive Maintenance In Smart Grids	(Omol et al., 2024)	Isolation Forest One-Class SVM, Autoencoder, Random Forest	Machine learning terbukti efektif mendeteksi anomali pada data sensor IoT dan sangat mendukung <i>predictive maintenance</i> pada smart grid.
5.	<i>Federated Learning-Based Anomaly Detection with Isolation Forest in the IoT-Edge Continuum</i>	Xiang et al., 2024	Federated Learning (FL) , Isolation Forest (iForest) , Layer-by-Layer Federated Tree Training (FL-Tree)	FLiForest berhasil memberikan akurasi deteksi anomali tertinggi dibanding iForest, L2SH, OptiForest, dan DiForest sebagian besar dari 10 dataset yang diuji. Model tetap efektif, stabil, dan cepat, serta menjaga privasi data karena pelatihan tanpa berbagi data mentah.

Berdasarkan **Tabel 2**, dapat disimpulkan bahwa penelitian mengenai Federated Learning untuk deteksi anomali telah dilakukan pada berbagai domain, terutama data terdistribusi seperti sistem IoT, jaringan, dan deret waktu. Penelitian Liu et al. (2022) menunjukkan bahwa Federated Learning dapat memberikan performa deteksi anomali yang kompetitif dengan model terpusat, sehingga cocok diterapkan pada data time series yang berasal dari banyak node. Penelitian Cavallin & Mayer (2022) juga menguatkan temuan tersebut, di mana beberapa pendekatan federated mampu mempertahankan akurasi tinggi meskipun data tersebar di banyak klien, meskipun performa dapat menurun pada distribusi data yang tidak seimbang.

Selanjutnya, penelitian Wei et al. (2022) membuktikan bahwa Autoencoder sangat efektif untuk mendeteksi anomali pada data sensor time series melalui pendekatan rekonstruksi. Meskipun penelitian tersebut belum menggunakan Federated Learning, hasilnya menunjukkan bahwa model AE merupakan metode yang tepat untuk mempelajari pola normal pada data berurutan.

Dua penelitian terbaru juga memberikan kontribusi penting. Penelitian Omol et al. (2024) menunjukkan bahwa berbagai algoritma machine learning seperti Isolation Forest, One-Class SVM, Autoencoder, dan Random Forest mampu mendeteksi anomali pada data sensor IoT untuk keperluan predictive maintenance pada smart grid secara efektif. Sementara itu, penelitian Xiang et al. (2024) melalui FLiForest membuktikan bahwa integrasi Federated Learning dengan Isolation Forest dapat menghasilkan akurasi deteksi yang lebih tinggi dibandingkan metode baseline, sekaligus menjaga privasi data karena pelatihan dilakukan tanpa berbagi data mentah.

Dari keseluruhan penelitian tersebut, terlihat bahwa penerapan Federated Learning dalam deteksi anomali telah mengalami perkembangan signifikan, namun sebagian besar masih berfokus pada domain industri, keamanan jaringan, smart grid, serta kualitas udara. Penelitian Omol et al. (2024) menunjukkan efektivitas berbagai algoritma machine learning dalam mendeteksi anomali pada data sensor IoT untuk kebutuhan predictive maintenance, sedangkan penelitian Xiang et al. (2024) memperlihatkan bahwa integrasi Federated Learning dengan Isolation Forest mampu meningkatkan akurasi deteksi sekaligus menjaga privasi data di lingkungan IoT-edge. Meskipun demikian, penerapan Federated Learning untuk data sensor IoT pertanian, khususnya tanaman tomat, masih sangat terbatas. Selain itu, belum banyak penelitian yang mengintegrasikan Autoencoder dalam skema Federated Learning untuk menangani permasalahan anomali pada data sensor pertanian yang bersifat time series dan terdistribusi.

Berdasarkan celah tersebut, penelitian ini berupaya mengembangkan model Federated Learning berbasis Autoencoder untuk mendeteksi anomali pada data sensor IoT pertanian tomat. Pendekatan ini diharapkan mampu meningkatkan akurasi deteksi anomali, menjaga kerahasiaan data sensor melalui pelatihan terdistribusi, serta mendukung implementasi pertanian presisi yang lebih efisien dan adaptif terhadap kondisi lapangan.

III. METODOLOGI PENELITIAN

3.1 Waktu dan Tempat Penelitian

Penelitian ini dilaksanakan di Program Studi Sistem Informasi, Jurusan Teknik Elektro dan Informatika, Fakultas Sains dan Teknologi, Universitas Jambi yang berlokasi di Jl. Jambi–Muaro Bulian KM 15, Mendalo Darat, Kabupaten Muaro Jambi. Kegiatan penelitian berlangsung mulai dari November 2025 hingga Maret 2026. Pada rentang waktu tersebut, seluruh tahapan mulai dari pengumpulan dataset sensor IoT, proses prapemrosesan data, perancangan model Federated Learning, pelatihan Autoencoder, hingga evaluasi deteksi anomali dilakukan secara sistematis untuk memastikan model yang dihasilkan valid dan dapat diimplementasikan.

3.2 Bahan dan Alat Penelitian

Penelitian ini memanfaatkan data sensor IoT yang berasal dari budidaya tanaman tomat dengan skema irigasi berbeda. Dataset tersebut digunakan sebagai sumber utama untuk proses prapemrosesan, pelatihan model, serta pengujian deteksi anomali. Informasi pendukung lainnya diperoleh melalui studi literatur, jurnal ilmiah, serta referensi daring yang relevan dengan topik Federated Learning, Autoencoder, dan sistem pemantauan pertanian berbasis IoT.

Alat yang di gunakan dalam penelitian ini, yaitu :

1. Perangkat Keras (Hardware)

Laptop Lenovo Thinkpat T480 dengan spesifikasi :

- a. Intel(R) Core(TM) i5-8350U CPU @ 1.70GHz 1.90 GHz
- b. SSD 256 GB
- c. RAM 16 GB, dan

2. Perangkat Lunak (Software)

- a. Sistem Operasi Windows 11 64 Bit
- b. Bahasa Pemrograman Python 3.10.0
- c. Library *Flwr (Flower)*, *Tensorflow*, *Pandas*, *Numpy*, *Sci-Kit-Learn*, *Maltpolib*, *Seaborn*, *Datetime*, *Os*, *Dan Glob*
- d. Google Colab

3.3 Jenis dan Sumber Data

Data yang digunakan dalam penelitian ini berasal dari dataset IoT pertanian tomat yang didistribusikan secara publik melalui platform Mendeley Data. Dataset tersebut berjudul *“IoT-based Data Collection in a Tomato Cultivation*

Under Different Irrigation Regimes” yang dikembangkan oleh Belli et al., (2025). Dataset ini merupakan hasil eksperimen lapangan pada budidaya tanaman tomat (*Solanum lycopersicum L. cv. HEINZ 1301*) di *Azienda Sperimentale Stuard*, Parma, Italia.

id	ts_generation	device_identifier	line	electrical_conductivity	humidity	temperature	battery
32299	1688047456101	9d5fe860849cee394ff52cd69a6d69642ea9f4756dac178e8ac998373f88563d	2	723	31.9	29.1	NULL
32309	1688048057491	9d5fe860849cee394ff52cd69a6d69642ea9f4756dac178e8ac998373f88563d	2	723	31.9	29	NULL
32319	1688048658856	9d5fe860849cee394ff52cd69a6d69642ea9f4756dac178e8ac998373f88563d	2	723	31.74	29	NULL
32329	1688049260229	9d5fe860849cee394ff52cd69a6d69642ea9f4756dac178e8ac998373f88563d	2	722	31.74	29	NULL
32348	1688050462965	9d5fe860849cee394ff52cd69a6d69642ea9f4756dac178e8ac998373f88563d	2	723	31.61	28.9	NULL
32358	1688051064328	9d5fe860849cee394ff52cd69a6d69642ea9f4756dac178e8ac998373f88563d	2	720	31.55	28.8	NULL
32368	1688051665689	9d5fe860849cee394ff52cd69a6d69642ea9f4756dac178e8ac998373f88563d	2	720	31.48	28.8	NULL
32378	1688052267071	9d5fe860849cee394ff52cd69a6d69642ea9f4756dac178e8ac998373f88563d	2	721	31.48	28.7	NULL

Gambar 3. Dataset Penelitian

Penelitian ini memanfaatkan dataset *stuard_soil_data.csv*, yaitu data yang dihasilkan oleh tiga perangkat sensor Milesight EM500 SMTc. Sensor tersebut merekam tiga parameter utama, yakni *electrical conductivity*, *humidity*, dan *temperature*. Dataset ini dipilih karena seluruh nilai yang terekam berasal langsung dari sensor lapangan, sehingga sangat sesuai untuk diterapkan pada skenario *anomaly detection* dalam penelitian ini.

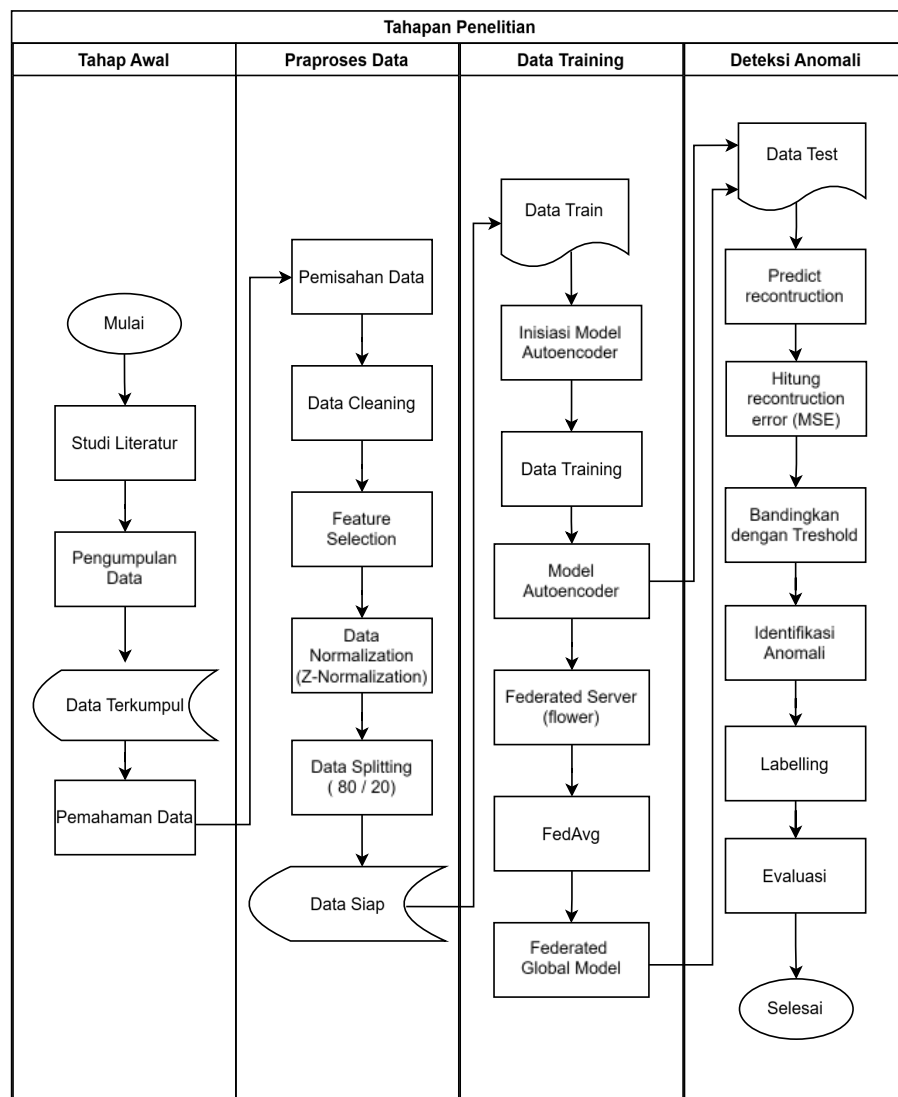
3.4 Tahapan Penelitian

Penelitian ini dilakukan melalui serangkaian langkah sistematis untuk membangun dan mengevaluasi model deteksi anomali pada data sensor IoT tanaman tomat. Proses penelitian diawali dengan pengumpulan data sensor serta pemahaman karakteristik dasar data, meliputi parameter *electrical conductivity*, *humidity*, dan *temperature*. Tahap selanjutnya adalah praproses data yang mencakup pembersihan data dan normalisasi untuk menyamakan skala antar fitur, sehingga data siap digunakan sebagai input model. Pada tahap ini, penentuan *threshold* awal juga dilakukan sebagai acuan dalam proses deteksi anomali.

Model deteksi anomali yang dikembangkan dalam penelitian ini menggunakan Autoencoder sebagai komponen utama untuk mempelajari pola normal data sensor. Autoencoder dilatih menggunakan data yang diasumsikan merepresentasikan kondisi normal, sehingga model mampu merekonstruksi data dengan baik apabila pola data masih berada dalam kondisi wajar. Proses pelatihan model dilakukan dalam skema *Federated Learning*, di mana setiap klien yang merepresentasikan perbedaan kondisi irigasi melakukan pelatihan model

secara lokal. Parameter model hasil pelatihan lokal selanjutnya digabungkan menggunakan metode Federated Averaging untuk membentuk model global tanpa harus memusatkan data mentah pada satu lokasi.

Tahap akhir penelitian difokuskan pada evaluasi performa model deteksi anomali. Evaluasi dilakukan dengan menganalisis reconstruction error yang dihitung menggunakan Mean Squared Error (MSE) per baris data, sebagai indikator tingkat penyimpangan nilai sensor terhadap pola normal yang telah dipelajari model. Hasil deteksi anomali berbasis MSE selanjutnya divalidasi menggunakan threshold fisiologis tanaman tomat untuk memastikan bahwa anomali yang terdeteksi tidak hanya menyimpang secara statistik, tetapi juga memiliki relevansi biologis. Hasil evaluasi ini digunakan untuk menilai keandalan model dalam mendeteksi anomali pada data sensor IoT tanaman tomat.



Gambar 4. Tahapan Penelitian

Gambar 4 menunjukkan alur lengkap tahapan penelitian yang dilakukan, mulai dari tahap awal pengumpulan dan pemahaman data hingga proses pelatihan model dalam skema Federated Learning. Setiap tahapan saling terhubung dan membentuk alur sistematis yang memastikan pembangunan model deteksi anomali dilakukan secara terstruktur dan konsisten. Tahap awal difokuskan pada identifikasi kebutuhan penelitian serta pemahaman karakteristik data sensor IoT tanaman tomat, yang meliputi parameter electrical conductivity, humidity, dan temperature.

Tahap praproses data bertujuan untuk mempersiapkan data agar siap digunakan sebagai input model. Selanjutnya, model Autoencoder dilatih secara lokal pada masing-masing klien menggunakan data yang tersedia tanpa memindahkan data mentah ke pusat. Parameter model hasil pelatihan lokal kemudian digabungkan menggunakan metode Federated Averaging untuk membentuk model global. Model global inilah yang digunakan pada tahap selanjutnya untuk melakukan rekonstruksi data dan mendeteksi anomali berdasarkan nilai reconstruction error. Alur ini memberikan gambaran utuh mengenai bagaimana penelitian dilaksanakan secara end-to-end hingga model akhir siap dievaluasi dalam mendeteksi anomali pada data sensor IoT tanaman tomat.

3.5 Tahapan Pendahuluan

Pada tahap awal penelitian, peneliti melakukan studi literatur untuk memperkuat landasan teori terkait IoT, deteksi anomali, model Autoencoder, serta pendekatan Federated Learning. Proses ini juga mencakup analisis komparatif terhadap berbagai penelitian sebelumnya guna mengidentifikasi kebaruan yang dapat diterapkan dalam penelitian ini. Adapun tabel berikut menyajikan kelebihan dan kekurangan masing-masing penelitian sebagai dasar pembeda dan penguatan kontribusi penelitian ini.

Tabel 3. Perbandingan penelitian

Peneliti	Dataset	Encoder / Model	Federated Learning	Anomaly Detect
Rizki, 2025	Mendeley Data	Autoencoder	FedAvg	Unsupervised , Reconstruction error + Threshold-based anomaly detection
Liu et al., 2022	SMD, SMAP, PSM	DeepSVDD, LSTM-AE, USAD, GDN, TranAD	FedAvg, FedProx, SCAFFOLD, MOON	Recontruction error model

Cavallin & Mayer, 2022	Kaggle	MLP, GMM, IF	Federated MLP, Federated GMM, Federated iForest	Supervised, semi-supervised, unsupervised
Wei et al., 2022	Dunedin CO ₂ time-series dataset	LSTM Autoencoder	xxx	Reconstruction error + Threshold-based detection
Omol et al., 2024	xxx	Isolation Forest, One-Class SVM, Autoencoder, Random Forest	xxx	Unsupervised & supervised anomaly detection
Xiang et al., 2024	UCI & AD Repository	Isolation Forest (iForest) + OPTiForest + LSHiForest + DiFores	FLiForest	Isolation Forest

Setelah melakukan kajian tersebut, diketahui bahwa penelitian-penelitian sebelumnya memiliki fokus dan karakteristik yang beragam. Misalnya, Liu et al. (2022) mengembangkan benchmark federated anomaly detection dengan berbagai model encoder, tetapi tidak secara spesifik menguji penerapannya pada domain pertanian. Cavallin & Mayer (2022) menyoroti performa algoritma FL pada data fraud dan medis, namun metode yang diuji tidak memanfaatkan model berbasis urutan seperti LSTM. Penelitian Wei et al. (2022) berfokus pada LSTM Autoencoder untuk kualitas udara, tetapi tidak menerapkan federated learning. Sementara itu, penelitian Omol et al. (2024) membahas teknik ML secara umum, dan Xiang et al. (2024) mengusulkan FL-iForest yang berbasis struktur pohon, bukan time series deep learning.

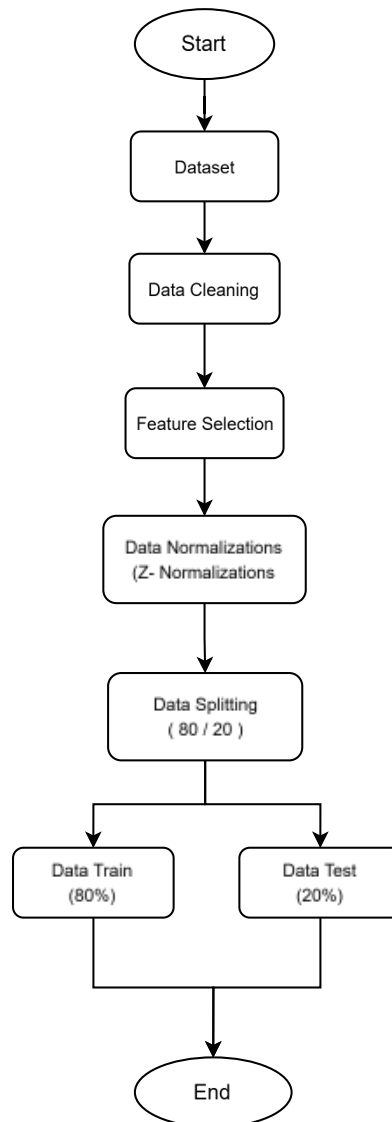
Dengan membandingkan penelitian-penelitian tersebut, terlihat bahwa penelitian ini memiliki kelebihan sekaligus mengisi celah penelitian yang belum terjawab sebelumnya. Penelitian ini menggabungkan Federated Learning dengan model Autoencoder, yang secara khusus dirancang untuk data sensor pertanian tomat yang bersifat time series, dan menambahkan mekanisme threshold-based anomaly detection berbasis kondisi fisiologis tanaman serta rekonstruksi model. Kajian literatur ini membantu memastikan bahwa metode yang digunakan relevan dengan kebutuhan penelitian dan sesuai dengan perkembangan riset terkini.

Setelah itu, peneliti memasuki tahap *data understanding*, yaitu proses memahami karakteristik dan struktur dataset sensor IoT tanaman tomat yang digunakan. Tahapan ini mencakup identifikasi jenis variabel yang tersedia seperti suhu tanah, kelembaban tanah, dan electrical conductivity serta memahami pola

pencatatan data, frekuensi pengukuran, dan kondisi tiap line irigasi (100%, 60%, dan 30%).

3.6 Preprocessing Data

Setelah proses *data understanding* selesai dan karakteristik dataset sensor IoT telah diketahui, langkah berikutnya adalah melakukan *preprocessing data*. Tahap ini merupakan langkah penting untuk memastikan bahwa data berada dalam kondisi yang bersih, konsisten, dan siap digunakan dalam pelatihan model Autoencoder maupun skema Federated Learning. Mengingat data sensor IoT sering mengandung ketidakraturan seperti nilai kosong, pencatatan tidak wajar, perbedaan satuan, hingga potensi *noise*, maka serangkaian proses *preprocessing* harus dilakukan agar model dapat belajar secara optimal.



Gambar 5. Tahapan pre-processing data

Gambar 5 menunjukkan alur preprocessing data yang diterapkan pada penelitian ini. Proses dimulai dari pengumpulan dataset sensor, kemudian dilanjutkan dengan tahap data cleaning untuk memastikan format dan kualitas data telah valid. Setelah itu dilakukan feature selection dengan memilih variabel sensor yang relevan, diikuti proses normalisasi menggunakan Z-normalization untuk menyamakan skala antar fitur.

Selanjutnya, data dibagi menjadi data latih dan data uji dengan proporsi 80% dan 20%. Data latih digunakan untuk proses pembelajaran model Autoencoder, sedangkan data uji disiapkan untuk tahap evaluasi dan deteksi anomali. Rangkaian preprocessing ini memastikan data berada dalam kondisi yang konsisten dan siap digunakan pada tahapan pemodelan berikutnya.

3.6.1 Pemisahan Data

Pada proses ini keseluruhan data yang semula tergabung dalam satu berkas CSV dipisahkan menjadi tiga *client* sesuai dengan sistem irigasi yang digunakan, yaitu Line 1, Line 2, dan Line 3. Proses pemisahan ini penting untuk mencerminkan karakteristik masing-masing lahan yang memiliki perlakuan irigasi berbeda, sehingga setiap *client* dapat dilatih secara independen dalam skenario *federated learning*.

id	ts_generation	device_identifier	line	electrical_conductiv	humidity	temperature	battery
31599	1.69E+17	91738f0250e567c5c	1	373	30.78	24.8	
31606	1.69E+17	91738f0250e567c5c	1	372	30.78	24.8	
31613	1.69E+17	91738f0250e567c5c	1	371	30.71	24.7	
31620	1.69E+17	91738f0250e567c5c	1	377	30.71	24.7	
31627	1.69E+17	91738f0250e567c5c	1	374	30.64	24.6	
31634	1.69E+17	91738f0250e567c5c	1	372	30.57	24.6	
31639	1.69E+17	91738f0250e567c5c	1	369	30.57	24.6	
31646	1.69E+17	91738f0250e567c5c	1	368	30.5	24.5	
31653	1.69E+17	91738f0250e567c5c	1	366	30.42	24.5	
31660	1.69E+15	91738f0250e567c5c	1	366	30.42	24.5	
id	ts_generation	device_identifier	line	electrical_conductiv	humidity	temperature	battery
32299	1.69E+17	9d5fe860849cee394	2	723	31.9	29.1	
32309	1.69E+17	9d5fe860849cee394	2	723	31.9	29	
32319	1.69E+17	9d5fe860849cee394	2	723	31.74	29	
32329	1.69E+17	9d5fe860849cee394	2	722	31.74	29	
32348	1.69E+17	9d5fe860849cee394	2	723	31.61	28.9	
32358	1.69E+17	9d5fe860849cee394	2	720	31.55	28.8	
32368	1.69E+17	9d5fe860849cee394	2	720	31.48	28.8	
32378	1.69E+17	9d5fe860849cee394	2	721	31.48	28.7	
32390	1.69E+17	9d5fe860849cee394	2	722	31.41	28.7	
32401	1.69E+17	9d5fe860849cee394	2	722	31.41	28.7	
id	ts_generation	device_identifier	line	electrical_conductiv	humidity	temperature	battery
32424	1.69E+17	0fa420b662607665e	3	1022	44.38	27.7	
32434	1.69E+17	0fa420b662607665e	3	1023	44.29	27.6	
32443	1.69E+17	0fa420b662607665e	3	1024	44.29	27.6	
32452	1.69E+17	0fa420b662607665e	3	1024	44.29	27.5	
32461	1.69E+17	0fa420b662607665e	3	1025	44.18	27.5	
32470	1.69E+17	0fa420b662607665e	3	1025	44.18	27.4	
32479	1.69E+17	0fa420b662607665e	3	1026	44.18	27.3	
32487	1.69E+17	0fa420b662607665e	3	1027	43.99	27.3	
32496	1.69E+17	0fa420b662607665e	3	1028	43.99	27.2	
32505	1.69E+17	0fa420b662607665e	3	1028	43.99	27.2	

Gambar 6. Dataset sebelum pemisahan

Data awal menunjukkan bahwa setiap line irigasi memiliki karakteristik dan pola sensor yang berbeda. Perbedaan ini mencerminkan variasi kondisi lingkungan pada tiap tingkat irigasi, sehingga diperlukan pemisahan data per line agar model dapat belajar pola normal yang sesuai dengan kondisi masing-masing klien.

Tabel 4. Total data setelah pemisahan

Client	Total Baris Data
Line 1	10966
Line 2	10862
Line 3	10838

Data dipisahkan menjadi tiga *client* berdasarkan tiga *line* irigasi yang berbeda, yaitu Line 1 (100%), Line 2 (60%), dan Line 3 (30%). Pemisahan ini bertujuan untuk merepresentasikan kondisi nyata di lapangan, di mana setiap line irigasi memiliki karakteristik lingkungan dan pola sensor yang berbeda. Line 1 memiliki jumlah data paling banyak, yaitu 10.966 baris, diikuti Line 2 dengan 10.862 baris, dan Line 3 dengan 10.838 baris.

Perbedaan jumlah baris data ini mencerminkan variasi pencatatan sensor serta kondisi operasional masing-masing lahan. Pemisahan berdasarkan line sangat penting dalam skema Federated Learning karena memungkinkan setiap *client* untuk melatih model secara independen sesuai karakteristik datanya, tanpa harus menggabungkan data secara terpusat. Dengan demikian, model mampu mempertahankan privasi data sekaligus menangkap anomali spesifik pada setiap perlakuan irigasi.

3.6.2 Data Cleaning

Pada tahap *data cleaning*, peneliti melakukan konversi atribut *ts_generation* menjadi format timestamp waktu sebenarnya. Konversi ini penting untuk memastikan bahwa data dapat diproses secara kronologis dalam analisis deret waktu serta cocok untuk pelatihan model Autoencoder. Berikut merupakan data sebelum dilakukan proses cleaning, di mana nilai *ts_generation* masih tercatat dalam format UNIX timestamp:

id	ts_generation	device_identifier	line	electrical_conductivity	humidity	temperature	battery
32424	1.69E+17	0fa420b6626076658	3	1022	44.38	27.7	
32434	1.69E+17	0fa420b6626076658	3	1023	44.29	27.6	
32443	1.69E+17	0fa420b6626076658	3	1024	44.29	27.6	
32452	1.69E+17	0fa420b6626076658	3	1024	44.29	27.5	
32461	1.69E+17	0fa420b6626076658	3	1025	44.18	27.5	
32470	1.69E+17	0fa420b6626076658	3	1025	44.18	27.4	
32479	1.69E+17	0fa420b6626076658	3	1026	44.18	27.3	
32487	1.69E+17	0fa420b6626076658	3	1027	43.99	27.3	
32496	1.69E+17	0fa420b6626076658	3	1028	43.99	27.2	

Gambar 7. Data sebelum cleaning

Format tersebut belum dapat menggambarkan waktu pencatatan sensor secara jelas sehingga memerlukan proses transformasi. Setelah dilakukan konversi, atribut ts_generation telah berubah menjadi format waktu sebenarnya (YYYY-MM-DD HH:MM:SS).

A	B	C	D	E	F	G	H
id	timestamp	device_identifier	line	electrical_conductivity	humidity	temperature	battery
32424	2023-06-29 14:46:40	0fa420b6626076658	3	1022	44.38	27.7	
32566	2023-06-29 17:33:20	0fa420b6626076658	3	1033	43.79	26.7	
32557	2023-06-29 17:33:20	0fa420b6626076658	3	1032	43.79	26.8	
32548	2023-06-29 17:33:20	0fa420b6626076658	3	1031	43.89	26.9	
32539	2023-06-29 17:33:20	0fa420b6626076658	3	1031	43.79	26.9	
32513	2023-06-29 17:33:20	0fa420b6626076658	3	1030	43.99	27.1	
32505	2023-06-29 17:33:20	0fa420b6626076658	3	1028	43.99	27.2	
32496	2023-06-29 17:33:20	0fa420b6626076658	3	1028	43.99	27.2	
32522	2023-06-29 17:33:20	0fa420b6626076658	3	1030	43.99	27.1	
32479	2023-06-29 17:33:20	0fa420b6626076658	3	1026	44.18	27.3	

Gambar 8. Data setelah cleaning

Setelah proses konversi dan pengurutan data dilakukan, dapat dilihat pada Gambar 8 ditemukan bahwa pada beberapa waktu tertentu terdapat lebih dari satu data yang tercatat pada timestamp yang sama. Kondisi ini menunjukkan bahwa timestamp pada dataset merepresentasikan waktu pengiriman data yang telah dikumpulkan, bukan waktu pengukuran sensor secara individual.

Tabel 5 berikut sebagian distribusi jumlah data berdasarkan waktu pengiriman :

Tabel 5. Distribusi jumlah data per-timestamp

	Timestamp	Jumlah data
1	28/06/2023 22:06	9
2	29/06/2023 00:53	16
3	29/06/2023 14:46	30
4	29/06/2023 17:33	48
5	29/06/2023 20:20	50
6	29/06/2023 23:06	50

Berdasarkan hasil pengelompokan tersebut, terdapat 666 timestamp unik, di mana setiap timestamp memiliki jumlah data yang bervariasi, menunjukkan bahwa pada waktu tertentu data dikirim secara bersamaan dalam satu batch. Timestamp pada penelitian ini digunakan sebagai penanda urutan data (ordering) dan bukan sebagai representasi waktu pengukuran yang presisi. Oleh karena itu, data tetap dipertahankan tanpa penghapusan nilai timestamp yang duplikat, selama urutan data tetap konsisten dan tidak mengganggu proses analisis. Dengan demikian, hasil data cleaning memastikan data berada dalam format yang valid, teratur, dan siap digunakan pada tahap preprocessing serta pelatihan model Autoencoder.

3.6.3 Feature Selection

Feature selection bertujuan memilih variabel yang paling relevan untuk proses deteksi anomali. Pada dataset IoT tanaman tomat, tidak semua fitur diperlukan, sehingga hanya fitur penting yang berkaitan langsung dengan kondisi tanah dipilih. Fitur yang digunakan yaitu :

- Electrical Conductivity ($\mu\text{S}/\text{cm}$)
- Temperature ($^{\circ}\text{C}$)
- Humidity (%)

3.6.4 Data Normalization

Normalisasi dilakukan untuk menyamakan skala antar fitur sehingga model Autoencoder dapat mempelajari data secara stabil. Setiap fitur sensor memiliki rentang nilai yang berbeda misalnya suhu $10 - 40^{\circ}\text{C}$, kelembaban $10 - 60\%$, dan electrical conductivity (EC) $300 - 2000 \mu\text{S}/\text{cm}$ yang dapat menyebabkan perbedaan skala yang cukup besar. Untuk menghindari dominasi satu fitur terhadap fitur lainnya, diperlukan proses normalisasi agar seluruh variabel berada pada skala yang setara.

Dalam penelitian ini digunakan Z-Normalization (Standardization). Berikut adalah rumus Z-Normalization (Standardization) yang digunakan untuk menormalkan setiap fitur:

$$Z = \frac{X - \mu}{\sigma} \quad (1)$$

Keterangan :

X = Nilai asli suatu fitur

μ = rata - rata (mean) fitur tersebut

σ = standar deviasi fitur tersebut

Z = nilai hasil normalisasi

Proses normalisasi diterapkan pada seluruh fitur sensor untuk memastikan setiap variabel memiliki skala yang setara sebelum dimasukkan ke dalam model Autoencoder berbasis feedforward. Metode Z-Normalization dipilih karena mampu menangani karakteristik data sensor yang bersifat fluktuatif, namun tetap memiliki distribusi nilai yang relatif stabil. Dengan menghilangkan perbedaan skala antar fitur, model dapat mempelajari pola nilai normal secara lebih efektif tanpa dipengaruhi oleh dominasi fitur tertentu. Selain itu, Z-Normalization relatif tahan terhadap variasi nilai ekstrem dibandingkan metode normalisasi berbasis rentang, sehingga lebih sesuai untuk data sensor IoT yang rentan mengalami perubahan mendadak akibat kondisi lingkungan atau gangguan teknis. Proses normalisasi ini berperan penting dalam meningkatkan stabilitas proses pelatihan dan akurasi deteksi anomali berbasis reconstruction error pada model Autoencoder.

3.6.5 Data Splitting

Pada tahap data splitting, dataset dibagi menjadi data latih (train) dan data uji (test) dengan proporsi 80% untuk data latih dan 20% untuk data uji. Pembagian ini dilakukan setelah seluruh data melalui tahap data cleaning dan berada dalam urutan yang konsisten, sehingga pemisahan data dapat dilakukan secara sistematis tanpa mengganggu struktur data yang telah disiapkan. Tujuan utama dari pembagian ini adalah menyediakan data latih yang cukup besar agar model Autoencoder dapat mempelajari pola distribusi data normal secara optimal, sekaligus menyediakan data uji yang representatif untuk proses evaluasi.

Proporsi 80% data latih dan 20% data uji dipilih karena merupakan praktik yang umum digunakan dalam penelitian berbasis machine learning dan dinilai mampu menjaga keseimbangan antara proses pembelajaran model dan pengujian kinerja. Dengan porsi data latih yang lebih dominan, model memiliki kesempatan yang lebih baik untuk mengenali karakteristik data normal, sementara data uji digunakan untuk menilai kemampuan generalisasi model dalam mendeteksi penyimpangan atau anomali pada data yang belum digunakan selama pelatihan. Pendekatan ini memastikan bahwa evaluasi model dilakukan secara objektif dan mencerminkan kondisi penggunaan model pada data baru.

Tabel 6. Pembagian data

Data Sensor	Kelas	Pembagian Data Train dan Data Test		Jumlah Data Train & Test
		Data Train	Data Test	
		80%	20%	
32666	Electrical Conductivity (Client 1)	8772	2194	10966
	Humidity (Client 1)			
	Temprature (Client 1)			
	Electrical Conductivity (Client 2)	8689	2173	10862
	Humidity (Client 2)			
	Temprature (Client 2)			
	Electrical Conductivity (Client 3)	8670	2168	10838
	Humidity (Client 3)			
	Temprature (Client 3)			
Total		26131	6535	32666

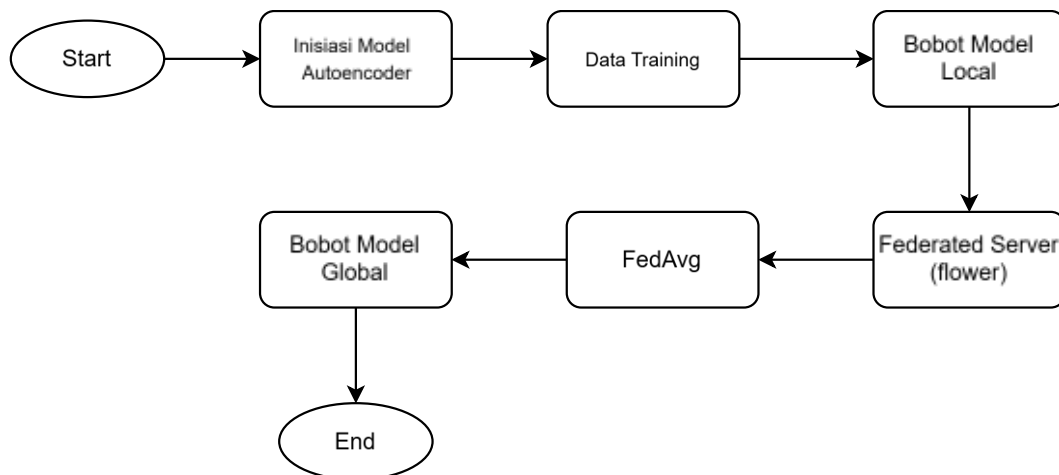
Tabel 6 menunjukkan hasil pembagian data pada setiap klien dan jenis sensor berdasarkan proporsi 80% data latih dan 20% data uji. Pembagian dilakukan secara konsisten pada masing-masing klien sehingga setiap model Autoencoder dilatih menggunakan data latih yang berasal dari klien yang sama, tanpa mencampurkan data antar klien. Dengan pendekatan ini, karakteristik distribusi data pada setiap klien tetap terjaga, sementara data uji digunakan untuk mengukur kemampuan model dalam mengenali pola normal dan mendeteksi penyimpangan berdasarkan reconstruction error. Secara keseluruhan, pembagian data ini memastikan proses pelatihan dan pengujian model dilakukan secara adil dan representatif terhadap kondisi data sebenarnya.

3.7 Data Training

Pada tahap ini, penelitian melakukan proses pelatihan model menggunakan data sensor IoT yang telah melalui tahap praproses pada masing-masing klien. Setiap klien (Line 1, Line 2, dan Line 3) melatih model Autoencoder berbasis feedforward secara independen menggunakan data lokal yang dimiliki. Proses ini memungkinkan model mempelajari karakteristik distribusi nilai sensor pada masing-masing klien yang merepresentasikan kondisi irigasi yang berbeda, tanpa harus memindahkan data mentah ke pusat.

Pelatihan model Autoencoder pada setiap klien bertujuan untuk mempelajari pola nilai normal data sensor, sehingga model mampu melakukan rekonstruksi data dengan baik pada kondisi wajar. Parameter atau bobot model hasil pelatihan lokal selanjutnya dikirimkan ke server pusat untuk digabungkan menggunakan metode Federated Averaging. Proses penggabungan ini menghasilkan model global yang merepresentasikan pengetahuan kolektif dari seluruh klien, sekaligus tetap menjaga privasi data pada masing-masing lokasi.

Untuk menggambarkan alur proses pelatihan data pada setiap klien hingga pembentukan model federated, berikut disajikan diagram alur proses training yang digunakan dalam penelitian ini.



Gambar 9. Alur data training

Pada Gambar 9 dapat di lihat seluruh alur kerja mulai dari proses training Autoencoder di setiap klien hingga pembentukan model global melalui Federated Averaging. Penjelasan rinci dari setiap tahapan pada flowchart tersebut disajikan pada subbab berikut.

3.7.1 Inisiasi Model Autoencoder

Pada tahap ini, setiap klien memulai proses pelatihan dengan menginisialisasi model Autoencoder berbasis *feedforward* yang memiliki arsitektur yang sama. Model dibangun menggunakan data sensor yang telah melalui proses normalisasi Z-score, sehingga seluruh klien memiliki struktur dan skala input yang konsisten sebelum pelatihan lokal dilakukan. Keseragaman arsitektur ini menjadi prasyarat penting dalam skema federated learning, karena memungkinkan proses penggabungan bobot model (*model aggregation*) dilakukan secara konsisten dan stabil pada tahap federasi.

Autoencoder yang digunakan terdiri atas dua komponen utama, yaitu *encoder* dan *decoder*. Bagian encoder berfungsi untuk memetakan data sensor berdimensi asli ke dalam representasi laten berdimensi lebih rendah (*latent space*), sedangkan bagian decoder bertugas merekonstruksi kembali data tersebut ke bentuk semula. Melalui proses ini, model diharapkan mampu mempelajari pola data normal yang terdapat pada sensor IoT pertanian tomat. Selanjutnya, perbedaan antara data asli dan hasil rekonstruksi (*reconstruction error*) digunakan sebagai dasar dalam proses deteksi anomali.

Ringkasan arsitektur Autoencoder yang digunakan dalam penelitian ini disajikan pada Tabel 7 berikut.

Tabel 7. *Autoencoder model summary*

Layer (Type)	Output Shape	Param #
input (InputLayer)	(None, 3)	0
enc_dense_1 (Dense)	(None, 8)	32
bottleneck (Dense)	(None, 4)	36
dec_dense_1 (Dense)	(None, 8)	40
output (Dense)	(None, 3)	27
Total params	/	135
Trainable params	/	135
Non_trainable params	/	0

Berdasarkan **Tabel 7**, model Autoencoder memiliki tiga neuron pada lapisan input yang merepresentasikan tiga fitur sensor utama, yaitu *Electrical Conductivity* (EC), *Humidity* (kelembapan tanah), dan *Temperature* (suhu). Pada bagian encoder, jumlah neuron dikurangi secara bertahap hingga mencapai lapisan *bottleneck* dengan empat neuron. Lapisan ini berperan sebagai representasi laten yang menyimpan informasi penting dari pola data normal dalam bentuk yang lebih ringkas.

Bagian decoder memiliki struktur yang simetris dengan encoder, di mana jumlah neuron ditingkatkan kembali hingga menghasilkan keluaran dengan dimensi yang sama seperti input. Seluruh parameter pada model bersifat *trainable*, yang memungkinkan proses pembelajaran berlangsung secara penuh selama pelatihan lokal di masing-masing klien. Dengan jumlah parameter yang relatif kecil, model ini tergolong ringan sehingga sesuai untuk diterapkan pada skenario federated learning yang melibatkan banyak klien dan keterbatasan sumber daya komputasi.

3.7.2 Data Training

Pada tahap data training, setiap klien (Client 1, Client 2, dan Client 3) menggunakan data sensor lokal yang telah melalui proses pra-proses dan normalisasi untuk melatih model Autoencoder secara terpisah. Pembagian klien ini merepresentasikan perbedaan kondisi irigasi dan karakteristik data pada masing-masing line sensor, sehingga setiap model lokal dapat mempelajari pola nilai normal yang spesifik sesuai dengan kondisi klien tersebut.

Data training diperoleh dari hasil data splitting, di mana 80% data digunakan sebagai data latih. Setiap baris data diperlakukan sebagai satu unit input, sehingga proses pelatihan difokuskan pada pembelajaran distribusi nilai normal data sensor pada setiap titik waktu. Selama proses pelatihan, masing-masing klien hanya menggunakan data lokalnya tanpa membagikan data mentah ke pihak lain, sehingga privasi data tetap terjaga. Hasil dari tahap data training ini berupa bobot model lokal (local weights) pada setiap klien, yang selanjutnya digunakan dalam proses agregasi Federated Learning untuk membentuk model global.

3.7.3 Pembentukan Model Lokal

Setelah proses pelatihan selesai, masing-masing klien menghasilkan sebuah model lokal yang disimpan dalam format file .h5. Penyimpanan model dilakukan untuk memastikan bahwa seluruh bobot hasil pelatihan dapat digunakan kembali pada tahap Federated Learning maupun proses evaluasi.

Secara teknis, penyimpanan dilakukan menggunakan function `model.save("client_1_model.h5")`, yang akan menyimpan arsitektur model beserta parameter pelatihannya dalam satu berkas. Dengan cara ini, model dapat di-load kembali ketika diperlukan, baik untuk proses agregasi global maupun untuk pengujian pada data uji.

Tabel 5. Model training autoencoder

Klien	Model Lokal
Klien 1	client_1_model.h5
Klien 2	client_2_model.h5
Klien 3	client_3_model.h5

Tahap pembentukan model lokal ini menghasilkan tiga file model .h5, masing-masing mewakili klien 1, klien 2, dan klien 3. Ketiga model tersebut kemudian dikirimkan ke Federated Server untuk menjalani proses agregasi menggunakan algoritma FedAvg, sehingga dihasilkan model global yang merepresentasikan pola umum dari seluruh klien.

3.7.4 Pengiriman Model ke Federated Server

Pada tahap ini, setiap klien mengirimkan bobot model lokal yang telah dihasilkan menuju Federated Server untuk dilakukan proses agregasi. Penelitian ini menggunakan framework Flower (FLWR) sebagai alat utama dalam mensimulasikan arsitektur Federated Learning. Flower memungkinkan proses federasi dijalankan tanpa infrastruktur jaringan yang kompleks, sehingga komunikasi antara klien dan server dilakukan dalam bentuk simulasi terkontrol, bukan implementasi distribusi nyata secara fisik.

Pengiriman model dalam Flower dilakukan melalui mekanisme built-in yang disebut `fit()` dan `get_parameters()`. Setelah proses pelatihan selesai di setiap klien, fungsi `get_parameters()` mengekstraksi bobot model lokal (local weights) dan menyiapkannya untuk dikirimkan ke server. Selanjutnya, bobot tersebut dikirimkan ke Federated Server dalam satu ronde federasi.

Federated Server kemudian menerima seluruh bobot model lokal dan menggabungkannya menggunakan algoritma Federated Averaging (FedAvg). Proses FedAvg menghitung rata-rata berbobot dari seluruh parameter model yang dikirimkan oleh klien, sehingga menghasilkan model global yang mewakili pola data kolektif dari seluruh klien.

Karena penelitian ini menggunakan simulasi, proses komunikasi klien-server tidak melibatkan protokol jaringan sebenarnya. Flower menjalankan seluruh klien dan server dalam satu lingkungan Python, namun tetap meniru alur kerja Federated Learning yang sesungguhnya, termasuk proses pengiriman bobot, agregasi, dan pembaruan model global. Dengan demikian, tahap ini memastikan bahwa setiap model lokal yang telah dilatih dikomunikasikan secara tepat ke server guna membentuk

model global yang akan digunakan pada tahap evaluasi akhir dan deteksi anomali.

3.7.5 Federated Averaging (FedAvg)

Setelah seluruh klien menyelesaikan pelatihan model Autoencoder secara lokal, dan mengirimkan model lokal ke server federated, langkah berikutnya adalah melakukan proses agregasi untuk menghasilkan model global. Agregasi ini merupakan inti dari mekanisme Federated Learning, di mana pembaruan model dilakukan tanpa memindahkan data mentah dari klien. Setiap klien hanya mengirimkan bobot model hasil pelatihannya, kemudian server pusat menggabungkan bobot-bobot tersebut menjadi satu model global yang mewakili pengetahuan kolektif dari seluruh klien. Untuk melakukan penggabungan ini, digunakan algoritma Federated Averaging (FedAvg) yang menggabungkan bobot lokal secara proporsional berdasarkan jumlah data yang dimiliki masing-masing klien.

Rumus agregasi:

$$w^{(t+1)} = \sum_{k=1}^K \frac{n_k}{N} w_k^{(t)} \quad (2)$$

Keterangan :

$w^{(t+1)}$: bobot model global pada iterasi berikutnya

$w_k^{(t)}$: bobot model lokal milik klien ke-k pada iterasi ke-t

n_k : jumlah data milik klien ke-k.

N : total data seluruh klien.

K : jumlah klien

Misalkan terdapat tiga klien (Line 1, Line 2, dan Line 3) yang masing-masing melakukan pelatihan model Autoencoder secara lokal. Setelah proses pelatihan selesai, setiap klien mengirimkan bobot model lokal kepada server federated. Bobot tersebut nantinya akan digabungkan menggunakan algoritma Federated Averaging (FedAvg).

Jumlah Data per Klien :

Klien 1 : 8772

Klien 2 : 8689

Klien 3 : 8670

Total data Klien :

$$N = n_1 + n_2 + n_3 = 8772 + 8689 + 8670 = 26131$$

Bobot model lokal $w_k^{(t)}$:

Klien 1 : 0,42

Klien 2 : 0,38

Klien 3 : 0,41

Setelah di dapatkan jumlah data dan bobot lokal dari hasil pelatihan masing masing klien tersebut, maka dapat dilanjutkan dengan :
Langkah 1 :

Langkah 1. Hitung Nilai $\frac{n_k}{N}$ untuk Setiap Klien

$$\frac{n_1}{N} = \frac{8772}{26131} = 0,3356$$

$$\frac{n_2}{N} = \frac{8689}{26131} = 0,3325$$

$$\frac{n_3}{N} = \frac{8670}{26131} = 0,3320$$

Langkah 2. Kalikan Setiap Fraksi dengan Bobot Lokal

$$\frac{n_1}{N} w_1^{(t)} = 0,3356 \times 0,42 = 0,1409$$

$$\frac{n_2}{N} w_2^{(t)} = 0,3325 \times 0,38 = 0,1263$$

$$\frac{n_3}{N} w_3^{(t)} = 0,3320 \times 0,41 = 0,1361$$

Langkah 3. Hitung Bobot Model Global

$$w^{(t+1)} = \frac{n_1}{N} w_1^{(t)} + \frac{n_2}{N} w_2^{(t)} + \frac{n_3}{N} w_3^{(t)}$$

$$w^{(t+1)} = 0,1409 + 0,1263 + 0,1361$$

Hasil Akhir :

$$w^{(t+1)} = 0,4033$$

Perhitungan di atas menggambarkan bagaimana algoritma Federated Averaging (FedAvg) menggabungkan bobot model lokal menjadi satu model global secara proporsional. Setiap bobot lokal dikalikan dengan rasio jumlah data klien terhadap total data seluruh klien, sehingga klien dengan data lebih besar memberikan pengaruh yang lebih kuat pada pembaruan model global. Pendekatan ini memastikan bahwa model global tetap mencerminkan karakteristik distribusi data secara keseluruhan tanpa memerlukan pemindahan data mentah, sehingga menjaga privasi dan mengurangi beban komunikasi. Bobot global yang dihasilkan selanjutnya

didistribusikan kembali ke klien untuk digunakan pada iterasi pelatihan berikutnya.

3.7.6 Model Global

Setelah proses agregasi FedAvg diselesaikan, Federated Server menghasilkan model global, yaitu model yang bobotnya merupakan hasil penggabungan (weighted averaging) dari seluruh bobot model lokal milik klien 1, klien 2, dan klien 3. Model global ini merepresentasikan pengetahuan gabungan dari ketiga klien, sehingga tidak hanya mencerminkan pola data lokal tertentu, melainkan pola umum dari seluruh variasi data sensor pada tiga line irigasi. Pada tahap ini, bobot model global kemudian dikembalikan (broadcast) kepada setiap klien sebagai versi model terkini yang siap digunakan.

Dalam penelitian ini, model global disimpan dalam bentuk file .npy, yang memuat seluruh parameter hasil agregasi FedAvg. Model global tersebut kemudian digunakan untuk mendeteksi anomali pada masing-masing klien dengan cara menghitung reconstruction error terhadap data uji mereka. Setiap prediksi rekonstruksi dari model global dibandingkan dengan data asli, dan selisihnya digunakan untuk menentukan apakah sebuah data masuk kategori normal atau anomali, berdasarkan threshold yang telah ditetapkan pada tahap sebelumnya. Dengan demikian, penelitian ini dapat membandingkan performa deteksi anomali antara model lokal dan model global pada setiap klien.

3.8 Threshold Anomali

Threshold anomali merupakan nilai batas yang digunakan untuk menentukan apakah suatu data hasil pengukuran sensor berada dalam kondisi normal atau termasuk sebagai anomali. Penentuan threshold menjadi tahap penting dalam proses deteksi anomali karena berfungsi sebagai acuan pengambilan keputusan terhadap hasil rekonstruksi data oleh model Autoencoder. Tanpa adanya threshold yang jelas, hasil deteksi anomali tidak dapat diinterpretasikan secara objektif dan konsisten.

Dalam penelitian ini, digunakan dua jenis threshold, yaitu threshold reconstruction error dan threshold fisiologis. Threshold reconstruction error ditentukan berdasarkan kinerja model Autoencoder dalam merekonstruksi data, sedangkan threshold fisiologis ditetapkan berdasarkan batas kondisi normal tanaman tomat menurut literatur agronomis. Penggunaan kedua threshold ini bertujuan untuk memastikan bahwa deteksi anomali tidak hanya bergantung

pada pola matematis yang dipelajari model, tetapi juga mempertimbangkan batasan biologis yang relevan dengan kondisi nyata tanaman.

3.8.1 Threshold Reconstruction Error

Threshold reconstruction error merupakan batas nilai kesalahan rekonstruksi yang digunakan untuk menentukan apakah suatu data hasil pengukuran sensor tergolong normal atau anomali berdasarkan kinerja model Autoencoder . Pada pendekatan autoencoder, deteksi anomali tidak dilakukan melalui proses klasifikasi secara langsung, melainkan dengan mengukur kemampuan model dalam merekonstruksi kembali data input. Model yang dilatih menggunakan data normal akan menghasilkan kesalahan rekonstruksi yang rendah ketika menerima data dengan pola yang serupa, dan menghasilkan kesalahan yang lebih tinggi ketika dihadapkan pada data yang menyimpang.

Threshold reconstruction error berfungsi sebagai batas maksimum kesalahan rekonstruksi yang masih dapat ditoleransi sebagai kondisi normal. Apabila nilai reconstruction error suatu data melebihi threshold yang telah ditetapkan, maka data tersebut dikategorikan sebagai anomali karena dianggap menyimpang dari pola nilai normal yang dipelajari oleh model.

Pada penelitian ini, reconstruction error dihitung menggunakan Mean Squared Error (MSE) karena metrik ini mampu mengukur perbedaan antara data asli dan data hasil rekonstruksi secara kuantitatif serta sensitif terhadap penyimpangan nilai yang besar. Nilai MSE dihitung per baris data, sehingga setiap observasi sensor dapat dievaluasi secara langsung.

Penentuan threshold dilakukan menggunakan pendekatan statistik berdasarkan distribusi nilai reconstruction error pada data latih. Nilai rata-rata (μ) dan standar deviasi (σ) dari reconstruction error digunakan sebagai dasar dalam menentukan threshold, yang merepresentasikan batas kesalahan rekonstruksi yang masih dianggap normal. Nilai reconstruction error yang melampaui batas tersebut selanjutnya dikategorikan sebagai anomali. Threshold reconstruction error ditentukan menggunakan rumus sebagai berikut:

$$T_{RE} = \mu + k \cdot \sigma \quad (3)$$

Keterangan:

T_{RE} :threshold reconstruction error,

μ : nilai rata-rata reconstruction error data training,

σ : standar deviasi reconstruction error data training,

k : konstanta pengali yang mengatur tingkat sensitivitas deteksi anomali.

Pada penelitian ini digunakan nilai konstanta $k = 3$ dalam penentuan threshold reconstruction error. Pemilihan nilai tersebut didasarkan pada prinsip three-sigma rule yang umum digunakan dalam analisis statistik untuk mengidentifikasi penyimpangan ekstrem, serta pertimbangan karakteristik data sensor pertanian yang memiliki fluktuasi alami. Pendekatan ini bertujuan untuk meminimalkan deteksi anomali palsu dan memastikan bahwa anomali yang teridentifikasi memiliki signifikansi statistik dan relevansi fisiologis.

Contoh Perhitungan :

Misalkan dari seluruh data training normal diperoleh:

$$\mu = 0.12 ,$$

$$\sigma = 0.02$$

$$k = 3$$

$$T_{RE} = \mu + k \cdot \sigma$$

$$T_{RE} = 0.12 + (3 \cdot 0.02)$$

$$T_{RE} = 0.18$$

Dengan demikian, setiap data yang memiliki nilai reconstruction error lebih besar dari 0,18 akan diklasifikasikan sebagai anomali, sedangkan data dengan nilai error di bawah atau sama dengan threshold tersebut dianggap sebagai kondisi normal.

Setelah threshold reconstruction error diperoleh, nilai tersebut digunakan pada tahap deteksi anomali terhadap data uji. Setiap data uji dihitung nilai reconstruction error-nya, kemudian dibandingkan dengan threshold yang telah ditentukan. Proses ini menghasilkan label awal berupa normal atau anomali berdasarkan penyimpangan pola temporal data.

Hasil deteksi berbasis threshold reconstruction error selanjutnya digunakan sebagai dasar untuk proses evaluasi model. Selain itu, dalam konteks penelitian ini, hasil deteksi tersebut juga akan diintegrasikan dengan threshold fisiologis tanaman tomat untuk memastikan bahwa anomali yang terdeteksi tidak hanya menyimpang secara statistik, tetapi juga memiliki relevansi biologis terhadap kondisi pertumbuhan tanaman.

3.8.2 Threshold Fisiologis Tomat

Penentuan threshold dilakukan untuk menetapkan batas normal dan anomali bagi tiga parameter utama data sensor, yaitu electrical conductivity (EC), soil humidity, dan soil temperature. Threshold ini berfungsi sebagai batas fisiologis tanaman tomat yang digunakan sebagai acuan dalam proses evaluasi hasil deteksi anomali, baik pada model lokal maupun model global. Dengan adanya threshold ini, setiap observasi pada data uji dapat dinyatakan “normal” atau “anomali” berdasarkan kesesuaiannya terhadap rentang nilai kondisi lingkungan yang secara fisiologis diperlukan tanaman tomat.

Penetapan threshold menggunakan dua pendekatan, yaitu (1) pendekatan agronomis, berdasarkan literatur fisiologi tanaman tomat (FAO, IFAS, dan jurnal hortikultura), serta (2) pendekatan irigasi, di mana setiap perlakuan irigasi (100%, 60%, 30%) memiliki rentang threshold berbeda karena ketersediaan air memengaruhi stabilitas suhu, kelembaban, dan salinitas tanah. Oleh sebab itu, threshold dibedakan berdasarkan kondisi irigasi untuk menghasilkan deteksi anomali yang kontekstual dan lebih akurat.

Tabel 8. Threshold Final Parameter Tanaman Tomat Berdasarkan Irigasi

Irigasi	Batas Data Normal		
	Electrical Conductivity ($\mu\text{S}/\text{cm}$)	Humidity (%)	Temperature ($^{\circ}\text{C}$)
100% (Client 1)	300 - 2000	20 - 45	15 - 35
60% (Client 2)	350 - 2300	18 - 40	14 - 36
30% (Client 3)	400 - 2600	15 - 35	13 - 37

Tabel 9. Threshold Final Parameter Tanaman Tomat Berdasarkan Irigasi

Threshold Anomaly		
Anomaly	Client	Measurement
EC too Low	Client 1	300 $\mu\text{S}/\text{cm}$
	Client 2	350 $\mu\text{S}/\text{cm}$
	Client 3	450 $\mu\text{S}/\text{cm}$

EC too High	Client 1	2000 $\mu\text{S}/\text{cm}$
	Client 2	2300 $\mu\text{S}/\text{cm}$
	Client 3	2600 $\mu\text{S}/\text{cm}$
Humidity too Low	Client 1	20%
	Client 2	18%
	Client 3	15%
Humidity too High	Client 1	45%
	Client 2	40%
	Client 3	35%
Temprature too Low	Client 1	15°C
	Client 2	14°C
	Client 3	13°C
Temprature too High	Client 1	35°C
	Client 2	36°C
	Client 3	37°C

Rentang electrical conductivity (EC) ditetapkan berdasarkan toleransi salinitas tanaman tomat yang berada pada kisaran 1.5–3.0 dS/m (1500–3000 $\mu\text{S}/\text{cm}$) sebagaimana dijelaskan oleh Cuartero dan Fernández-Muñoz (1999) serta Giordano et al. (2010). Kondisi lapangan pada proyek Stuard menunjukkan bahwa untuk irigasi penuh, nilai EC yang aman berada pada 300–2000 $\mu\text{S}/\text{cm}$, sedangkan pada irigasi defisit rentang tersebut meningkat secara bertahap. Hal ini terjadi karena pengurangan suplai air menurunkan proses leaching, sehingga akumulasi garam lebih mudah terjadi pada irigasi 60% dan 30%, menyebabkan batas atas EC menjadi lebih tinggi pada kedua kondisi tersebut.

Kondisi kelembaban tanah (soil humidity) juga menyesuaikan tingkat irigasi yang diterapkan. Tanaman tomat tumbuh optimal pada 60–80% kapasitas lapang, atau sekitar 25–45% soil moisture (FAO, 2020; Zotarelli et al., 2021). Untuk irigasi 100%, rentang normal ditetapkan 20–45%, sementara pada irigasi 60% dan 30% batas tersebut diturunkan menjadi masing-masing 18–40% dan 12–35%. Penurunan threshold ini konsisten dengan temuan IRRIFRAME yang menunjukkan bahwa kondisi irigasi defisit menyebabkan pergeseran kelembaban tanah menuju nilai yang lebih rendah akibat berkurangnya ketersediaan air.

Suhu tanah (soil temperature) ditetapkan berdasarkan rentang fisiologis tanaman tomat yang ideal untuk proses penyerapan nutrisi dan pertumbuhan akar, yaitu 18–30°C, dengan toleransi 15–35°C (Scholberg et al., 2000; FAO, 2005). Pada irigasi penuh, suhu cenderung stabil karena

kandungan air tanah yang tinggi memiliki efek buffer terhadap perubahan panas. Namun pada irigasi 60% dan 30%, fluktuasi suhu meningkat sehingga batas normal diperluas sekitar $\pm 1-2^{\circ}\text{C}$, menghasilkan threshold $14-36^{\circ}\text{C}$ dan $13-37^{\circ}\text{C}$. Penyesuaian ini diperlukan agar model deteksi anomali tetap realistis dalam menggambarkan kondisi variatif di lapangan.

Selain digunakan sebagai batas fisiologis lingkungan tanaman, threshold juga berfungsi sebagai komponen penting dalam proses evaluasi hasil deteksi anomali yang dilakukan oleh model Federated Learning Autoencoder. Pada tahap pelatihan, Autoencoder mempelajari pola normal dari data sensor termasuk hubungan temporal antarparameter seperti EC, humidity, dan temperature sehingga model mampu menghasilkan rekonstruksi yang akurat untuk data normal dan rekonstruksi yang buruk (error tinggi) untuk data yang tidak sesuai pola.

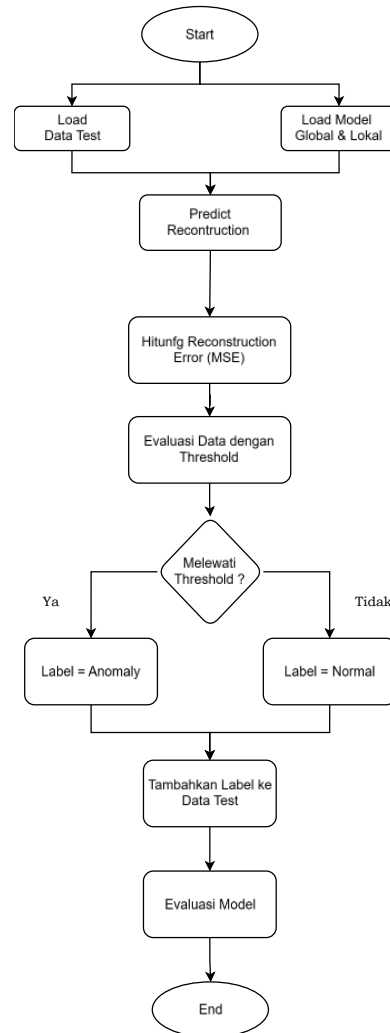
Setelah model dilatih, data uji direkonstruksi untuk menghasilkan *reconstruction error* yang merepresentasikan selisih antara nilai asli dan hasil rekonstruksi model. Nilai error ini digunakan sebagai dasar deteksi anomali berdasarkan ambang *reconstruction threshold* yang telah ditetapkan, kemudian divalidasi menggunakan threshold fisiologis tanaman tomat sebagai *ground-truth* agronomis. Pendekatan ini memastikan bahwa hasil deteksi anomali tidak hanya didasarkan pada pola matematis model, tetapi juga selaras dengan batasan biologis dan agronomis, sehingga interpretasi hasil menjadi lebih akurat dan dapat dipertanggungjawabkan secara ilmiah.

3.9 Deteksi Anomali

Tahap deteksi anomali merupakan tahapan akhir dalam penelitian ini yang bertujuan untuk mengidentifikasi kondisi data sensor tanah yang menyimpang dari pola normal. Proses deteksi anomali dilakukan dengan memanfaatkan model Autoencoder hasil pelatihan federated learning, baik model lokal pada masing-masing klien maupun model global, yang telah mempelajari distribusi data normal dari data latih.

Pada tahap ini, model Autoencoder tidak lagi melakukan proses pembelajaran, melainkan digunakan untuk mengevaluasi data uji (data test). Data uji direkonstruksi oleh model, kemudian dihitung nilai *reconstruction error* menggunakan Mean Squared Error (MSE) untuk setiap baris data. Nilai error tersebut selanjutnya dibandingkan dengan dua jenis threshold yang telah ditetapkan, yaitu *threshold reconstruction error* dan *threshold fisiologis tanaman*.

Data yang memiliki nilai error atau kondisi parameter di luar batas normal akan dikategorikan sebagai anomali, sedangkan data lainnya dikategorikan sebagai normal.



Gambar 10. Proses deteksi anomali

Gambar 10 menunjukkan alur proses deteksi anomali yang dimulai dari pemuatan data uji dan model Autoencoder (lokal dan global). Data uji kemudian direkonstruksi oleh model untuk memperoleh nilai reconstruction error. Nilai error tersebut dievaluasi menggunakan threshold yang telah ditentukan untuk memutuskan apakah suatu data melewati batas normal atau tidak. Apabila nilai error atau kondisi fisiologis melewati threshold, maka data diberi label anomali, sedangkan data yang berada dalam batas normal diberi label normal. Hasil pelabelan ini selanjutnya digunakan untuk mengevaluasi kinerja model dalam mendeteksi anomali pada data yang belum pernah dilihat sebelumnya.

3.9.1 Predict Recontruction

Pada tahap deteksi anomali, proses rekonstruksi data uji dilakukan melalui tahap prediksi (predict) menggunakan model Autoencoder yang telah dibangun dan dilatih pada tahap sebelumnya. Model yang digunakan terdiri dari dua jenis, yaitu model lokal Autoencoder dan model global Autoencoder hasil Federated Learning. Sebelum proses prediksi dilakukan, data uji serta kedua model tersebut terlebih dahulu dimuat (load) ke dalam sistem untuk memastikan kesiapan seluruh komponen.

Data uji yang digunakan merupakan data yang telah melalui tahap preprocessing, meliputi normalisasi dan pembentukan data dalam bentuk sequence. Data sequence tersebut kemudian diberikan sebagai input ke dalam model lokal dan model global menggunakan mekanisme prediksi untuk menghasilkan data hasil rekonstruksi. Output dari tahap ini berupa data hasil rekonstruksi yang memiliki struktur dan dimensi yang sama dengan data uji asli, sehingga memungkinkan dilakukan perbandingan secara langsung antara data asli dan data hasil prediksi rekonstruksi. Contoh hasil predict recontruction dapat di lihat pada tabel berikut.

Tabel 10. Contoh hasil *predict recontruction*

EC (Asli)	EC Rekonstruksi	Humidity (Asli)	Humidity Rekonstruksi	Temperature (Asli)	Temperature Rekonstruksi
373	372.9	30.78	30.76	24.8	24.79
372	372.4	30.78	30.75	24.8	24.78
371	371.3	30.71	30.70	24.7	24.71
377	376.6	30.71	30.69	24.7	24.68
374	373.7	50.42	50.22	24.6	24.61

Proses prediksi rekonstruksi dilakukan secara terpisah menggunakan model lokal dan model global. Hasil prediksi dari masing-masing model disimpan sebagai dasar untuk perhitungan reconstruction error pada tahap selanjutnya, serta untuk membandingkan karakteristik hasil rekonstruksi antara kedua model.

3.9.2 Recontruction Error

Reconstruction error merupakan ukuran yang digunakan untuk menilai sejauh mana model Autoencoder mampu merekonstruksi kembali data input yang merepresentasikan pola normal. Jika model telah mempelajari pola normal dengan baik, maka hasil rekonstruksi yang

dihasilkan akan sangat mendekati nilai data asli sehingga nilai error yang diperoleh relatif kecil. Sebaliknya, apabila data yang direkonstruksi memiliki perbedaan yang signifikan terhadap data asli, maka nilai error akan meningkat dan kondisi tersebut dapat diindikasikan sebagai anomali.

Pada penelitian ini, reconstruction error dihitung menggunakan Mean Squared Error (MSE), yaitu metode pengukuran kesalahan yang menghitung rata-rata kuadrat selisih antara data asli dan data hasil rekonstruksi. MSE dipilih karena mampu memberikan ukuran kuantitatif yang sensitif terhadap perbedaan nilai, terutama pada penyimpangan yang besar, sehingga efektif untuk mendeteksi perubahan pola data. Nilai MSE yang dihasilkan merepresentasikan tingkat ketidaksesuaian antara data uji dan pola normal yang telah dipelajari oleh model. Berikut rumus MSE yang di gunakan pada penelitian ini :

$$MSE = \frac{1}{n} (x_i - \hat{x}_i)^2 \quad (4)$$

Keterangan :

MSE : Mean Squared Error, yaitu nilai kesalahan rekonstruksi

x_i : nilai data sensor asli pada indeks ke-i

$\hat{x}_i$: nilai data hasil rekonstruksi ke-i

n : jumlah fitur

Contoh perhitungan menggunakan data contoh pada Tabel :

$$\begin{aligned} MSE &= \frac{1}{n} (x_i - \hat{x}_i)^2 \\ MSE &= \frac{1}{3} [(373 - 372.9)^2 + (30.78 - 30.76)^2 + (24.8 - 24.79)^2] \\ MSE &= \frac{1}{3} [(0.1)^2 + (0.02)^2 + (0.01)^2] \\ MSE &= \frac{1}{3} [0.01 + 0.0004 + 0.0001] \\ MSE &= \frac{1}{3} 0.0105 \\ MSE &= 0.0035 \end{aligned}$$

Berdasarkan contoh perhitungan yang telah dilakukan, diperoleh nilai Mean Squared Error (MSE) sebesar 0.0035. Nilai ini merupakan hasil rata-rata kuadrat selisih antara nilai data asli dan nilai hasil rekonstruksi pada tiga fitur sensor, yaitu konduktivitas listrik tanah, kelembapan tanah, dan suhu tanah. Nilai MSE tersebut digunakan sebagai contoh untuk

menggambarkan cara perhitungan reconstruction error pada satu data uji dalam penelitian ini.

Nilai reconstruction error berbasis MSE ini digunakan sebagai indikator utama dalam proses deteksi anomali. Dengan membandingkan nilai MSE terhadap threshold yang telah ditentukan, sistem dapat mengklasifikasikan data uji ke dalam kondisi normal atau anomali secara objektif. Dengan demikian, reconstruction error berperan penting sebagai dasar pengambilan keputusan dalam sistem deteksi anomali berbasis Autoencoder.

3.9.3 Deteksi Anomali dan Pelabelan Data

Pada tahap ini, proses deteksi anomali dilakukan dengan membandingkan nilai reconstruction error yang diperoleh dari perhitungan Mean Squared Error (MSE) per sequence dengan nilai threshold reconstruction error yang telah ditentukan sebelumnya. Nilai MSE per sequence merepresentasikan tingkat penyimpangan pola data uji terhadap pola normal yang telah dipelajari oleh model Autoencoder selama proses pelatihan.

Jika nilai MSE per sequence lebih kecil atau sama dengan threshold reconstruction error, maka data uji tersebut dikategorikan sebagai kondisi normal karena pola datanya masih sesuai dengan pola normal yang dikenali oleh model. Sebaliknya, apabila nilai MSE per sequence melebihi threshold reconstruction error, maka data uji dianggap mengalami penyimpangan pola dan diindikasikan sebagai anomali secara statistik.

Selanjutnya, hasil deteksi berbasis reconstruction error tersebut divalidasi menggunakan threshold fisiologis tanaman tomat. Pada tahap ini, nilai parameter sensor pada sequence yang terdeteksi menyimpang diperiksa apakah berada di dalam atau di luar batas fisiologis tanaman. Integrasi antara hasil deteksi berbasis MSE dan threshold fisiologis dilakukan untuk memastikan bahwa anomali yang terdeteksi tidak hanya menyimpang secara pola data, tetapi juga memiliki relevansi biologis terhadap kondisi pertumbuhan tanaman.

Berdasarkan hasil perbandingan tersebut, dilakukan proses pelabelan data sebagai keputusan akhir. Data uji diberi label normal apabila nilai MSE per sequence berada di bawah threshold reconstruction error dan nilai sensor masih berada dalam batas fisiologis tanaman. Sebaliknya, data uji diberi label anomali apabila nilai MSE per sequence melebihi threshold reconstruction error dan nilai sensor berada di luar

batas fisiologis. Hasil pelabelan ini menjadi keluaran akhir dari proses deteksi anomali dan digunakan sebagai dasar untuk analisis serta evaluasi lebih lanjut pada bab berikutnya.

Tabel 11. Contoh hasil deteksi

No	EC (Asli)	Humidity (Asli)	Temperature (Asli)	MSE	MSE > Threshold	Fisiologis	Label Akhir
1	373	30.78	24.8	0.0350	Tidak	Normal	Normal
2	372	30.78	24.8	0.0287	Tidak	Normal	Normal
3	371	30.71	24.7	0.0613	Ya	Normal	Normal
4	377	30.71	24.7	0.0425	Tidak	Normal	Normal
5	374	50.42	24.6	0.0894	Ya	Humidity too High	Anomali

Tabel 11 di atas disajikan sebagai contoh untuk menggambarkan alur proses deteksi anomali dan pelabelan data pada sistem yang diusulkan. Tabel ini menunjukkan bagaimana nilai Mean Squared Error (MSE) per sequence dibandingkan dengan threshold reconstruction error, kemudian divalidasi menggunakan threshold fisiologis tanaman tomat untuk menghasilkan label akhir berupa kondisi normal atau anomali. Contoh ini bertujuan untuk memperjelas mekanisme pengambilan keputusan dalam sistem deteksi anomali, bukan untuk merepresentasikan hasil pengujian atau evaluasi model yang sebenarnya. Hasil pengujian dan analisis kinerja model disajikan secara terpisah pada bab selanjutnya.

3.10 Evaluasi Model Federated Autoencoder

Evaluasi model dilakukan untuk menilai kinerja deteksi anomali pada sistem Federated Autoencoder yang dikembangkan. Evaluasi difokuskan pada perbandingan performa antara model lokal dan model global hasil agregasi federated learning menggunakan metode FedAvg. Meskipun model dikembangkan dengan pendekatan unsupervised learning, proses evaluasi dilakukan secara supervised dengan memanfaatkan label fisiologis sebagai ground truth guna mengukur kualitas hasil deteksi anomali secara objektif.

3.10.1 Skema Evaluasi Model Lokal dan Global

Evaluasi dilakukan menggunakan data uji yang tidak dilibatkan dalam proses pelatihan. Fokus evaluasi diarahkan pada hasil deteksi anomali yang dihasilkan oleh model Autoencoder lokal dan model global hasil Federated Learning. Performa kedua model diukur dengan membandingkan hasil deteksi terhadap label fisiologis sebagai ground

truth, sehingga dapat dianalisis perbedaan efektivitas dan konsistensi deteksi antara model lokal dan model global.

3.10.2 Confusion Matrix

Confusion matrix digunakan untuk mengevaluasi kinerja hasil deteksi anomali yang dihasilkan oleh model Autoencoder lokal dan model global hasil Federated Learning. Evaluasi ini dilakukan dengan membandingkan hasil deteksi masing-masing model terhadap label fisiologis sebagai ground truth, sehingga dapat diketahui tingkat ketepatan serta pola kesalahan deteksi yang dihasilkan oleh kedua model.

Pada confusion matrix, hasil deteksi diklasifikasikan ke dalam empat kategori, yaitu true positive (TP), true negative (TN), false positive (FP), dan false negative (FN). Kategori-kategori ini digunakan untuk menggambarkan hubungan antara hasil prediksi model dan kondisi aktual data uji.

Tabel 12. *Confusion Matrix*

	Prediksi Anomali	Prediksi Normal
Aktual Anomali	True Positive (TP)	False Negative (FN)
Aktual Normal	False Positive (FP)	True Negative (TN)

Secara konseptual, perbandingan confusion matrix antara model lokal dan model global bertujuan untuk melihat perbedaan jenis dan jumlah kesalahan deteksi yang dihasilkan oleh kedua model. Fokus utama analisis diarahkan pada false positive dan false negative, karena kedua jenis kesalahan tersebut menunjukkan kelemahan model dalam membedakan kondisi normal dan anomali.

Perbandingan confusion matrix digunakan untuk mengetahui apakah model global mampu mengurangi kesalahan deteksi, khususnya false positive dan false negative, yang sering muncul pada model lokal akibat bias data klien.

3.10.3 Precision, Recall, dan F1-Score

Setelah analisis confusion matrix, evaluasi dilanjutkan menggunakan metrik precision, recall, dan F1-score untuk mengukur kualitas hasil deteksi anomali yang dihasilkan oleh model Autoencoder lokal dan model global hasil Federated Learning. Ketiga metrik ini

digunakan karena mampu memberikan gambaran performa deteksi secara lebih komprehensif dibandingkan hanya melihat jumlah kesalahan deteksi.

Precision digunakan untuk menunjukkan tingkat keandalan model dalam mendeteksi anomali, yaitu seberapa besar proporsi data yang terdeteksi sebagai anomali benar-benar merupakan anomali berdasarkan label fisiologis. Recall digunakan untuk mengukur kemampuan model dalam menangkap seluruh kejadian anomali yang sebenarnya terjadi, sehingga berkaitan langsung dengan sensitivitas deteksi. Sementara itu, F1-score digunakan sebagai ukuran keseimbangan antara precision dan recall, sehingga dapat menggambarkan performa deteksi anomali secara keseluruhan.

Perhitungan nilai precision, recall, dan F1-score dilakukan berdasarkan nilai true positive, false positive, dan false negative yang diperoleh dari confusion matrix. Rumus yang digunakan dalam penelitian ini ditunjukkan sebagai berikut:

$$Precision = \frac{TP}{TP+FP} \quad (5)$$

$$Recall = \frac{TP}{TP+FN} \quad (6)$$

$$F1 - Score = \frac{2 \cdot (Precision \cdot Recall)}{Precision + Recall} \quad (7)$$

Rumus-rumus tersebut digunakan untuk menghitung metrik evaluasi secara terpisah pada model Autoencoder lokal dan model global hasil Federated Learning. Tabel berikut menyajikan format perbandingan metrik precision, recall, dan F1-score antara model lokal dan model global.

Tabel 13. Metrik evaluasi

Model	Precision	Recall	F1-Score
Autoencoder Lokal	Precision Lokal	Recall Lokal	F1 Lokal
Model Global	Precision Global	Recall Global	F1 Global

Tabel 13 tersebut digunakan untuk menunjukkan perbandingan performa deteksi anomali antara kedua model tanpa menampilkan nilai numerik secara eksplisit.

Secara konseptual, nilai precision yang lebih tinggi menunjukkan bahwa model memiliki tingkat kesalahan false positive yang lebih rendah, sedangkan nilai recall yang lebih tinggi menunjukkan kemampuan model dalam menangkap lebih banyak kejadian anomali. F1-score digunakan untuk menilai keseimbangan antara precision dan recall, sehingga memberikan gambaran performa deteksi secara menyeluruh.

Perbandingan metrik ini antara model lokal dan model global digunakan untuk menilai apakah penerapan Federated Learning mampu meningkatkan stabilitas dan efektivitas deteksi anomali dibandingkan dengan model lokal yang hanya dilatih menggunakan data klien tertentu.

3.10.4 Analisis Reconstruction Error Lokal dan Global

Evaluasi reconstruction error dilakukan untuk membandingkan kemampuan model Autoencoder lokal dan model global hasil Federated Learning dalam merekonstruksi data uji. Pada tahap ini, masing-masing model digunakan untuk merekonstruksi data uji yang sama, kemudian dihitung nilai reconstruction error sebagai indikator kesesuaian antara data asli dan hasil rekonstruksi model.

Output yang dihasilkan pada evaluasi ini berupa nilai reconstruction error pada data uji, yang selanjutnya diringkaskan dalam bentuk tabel atau grafik untuk memudahkan perbandingan antara model lokal dan model global. Ringkasan nilai reconstruction error tersebut digunakan untuk melihat tingkat kesalahan rekonstruksi serta kestabilan nilai error yang dihasilkan oleh masing-masing model. Melalui evaluasi ini, diharapkan model global hasil Federated Learning mampu menghasilkan reconstruction error yang lebih stabil dibandingkan model lokal.

DAFTAR PUSTAKA

- Adkisson, M., Kimmel, J. C., Gupta, M., & Abdelsalam, M. (2022). *Autoencoder-based Anomaly Detection in Smart Farming Ecosystem*. <http://arxiv.org/abs/2111.00099>
- Aijaz, N., Lan, H., Raza, T., Yaqub, M., Iqbal, R., & Pathan, M. S. (2025). Artificial intelligence in agriculture: Advancing crop productivity and sustainability. In *Journal of Agriculture and Food Research* (Vol. 20). Elsevier B.V. <https://doi.org/10.1016/j.jafr.2025.101762>
- Becker, S., Styp-Rekowski, K., Stoll, O. V. L., & Kao, O. (2022). *Federated Learning for Autoencoder-based Condition Monitoring in the Industrial Internet of Things*. <http://arxiv.org/abs/2211.07619>
- Belli, L., Davoli, L., Oddi, G., Preite, L., Galaverni, M., Ganino, T., & Ferrari, G. (2025). Smart agriculture dataset in a tomato cultivation under different irrigation regimes. *Data in Brief*, 60. <https://doi.org/10.1016/j.dib.2025.111521>
- Brendan McMahan Eider Moore Daniel Ramage Seth Hampson Blaise Agüera-Ag, H., & Arcas, A. (2017). *Communication-Efficient Learning of Deep Networks from Decentralized Data*.
- Cavallin, F., & Mayer, R. (2022). *Advanced Information Networking and Applications* (L. Barolli, F. Hussain, & T. Enokido, Eds.; Vol. 450). Springer International Publishing. <https://doi.org/10.1007/978-3-030-99587-4>
- Chen, S., & Guo, W. (2023). Auto-Encoders in Deep Learning—A Review with New Perspectives. In *Mathematics* (Vol. 11, Number 8). MDPI. <https://doi.org/10.3390/math11081777>
- Dembani, R., Karvelas, I., Akbar, N. A., Rizou, S., Tegolo, D., & Fountas, S. (2025). Agricultural data privacy and federated learning: A review of challenges and opportunities. *Computers and Electronics in Agriculture*, 232. <https://doi.org/10.1016/j.compag.2025.110048>
- Domínguez-Bolaño, T., Campos, O., Barral, V., Escudero, C. J., & García-Naya, J. A. (2022). An overview of IoT architectures, technologies, and existing open-source projects. In *Internet of Things (Netherlands)* (Vol. 20). Elsevier B.V. <https://doi.org/10.1016/j.iot.2022.100626>
- Hasan, M. R., Rahman, M. M., Shahriar, F., Khan, M. S. I., Mohi Uddin, K. M., & Hasan, M. M. (2024). Smart farming: Leveraging IoT and deep learning for sustainable tomato cultivation and pest management. *Crop Design*, 3(4). <https://doi.org/10.1016/j.crope.2024.100079>
- Iglesias, J. O., Perales, C. S., Geißler, S., Perino, D., & Lutu, A. (2025). *Anomaly Detection for IoT Global Connectivity*. <http://arxiv.org/abs/2508.09660>
- Krzysztoń, E., Rojek, I., & Mikołajewski, D. (2024). A Comparative Analysis of Anomaly Detection Methods in IoT Networks: An Experimental Study. In *Applied Sciences (Switzerland)* (Vol. 14, Number 24). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/app142411545>

- Lachekhab, F., Benzaoui, M., Tadjer, S. A., Bensmaine, A., & Hamma, H. (2024). LSTM-Autoencoder Deep Learning Model for Anomaly Detection in Electric Motor. *Energies*, 17(10). <https://doi.org/10.3390/en17102340>
- Laridi, S., Palmer, G., & Tam, K. M. M. (2024). Enhanced federated anomaly detection through autoencoders using summary statistics-based thresholding. *Scientific Reports*, 14(1). <https://doi.org/10.1038/s41598-024-76961-2>
- Lee, Y., Park, C., Kim, N., Ahn, J., & Jeong, J. (2024). LSTM-Autoencoder Based Anomaly Detection Using Vibration Data of Wind Turbines. *Sensors*, 24(9). <https://doi.org/10.3390/s24092833>
- Lim, L. H., Ong, L. Y., & Leow, M. C. (2025). Federated Learning for Anomaly Detection: A Systematic Review on Scalability, Adaptability, and Benchmarking Framework. In *Future Internet* (Vol. 17, Number 8). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/fi17080375>
- Liu, F., Zeng, C., Zhang, Le, Zhou, Y., Mu, Q., Zhang, Y., Zhang, Ling, & Zhu, C. (2022). *FedTADBench: Federated Time-Series Anomaly Detection Benchmark*. <http://arxiv.org/abs/2212.09518>
- Liu, Y., Zhang, L., Ge, N., & Li, G. (2022). *A Systematic Literature Review on Federated Learning: From A Model Quality Perspective*. <http://arxiv.org/abs/2012.01973>
- Mansoor, S., Iqbal, S., Popescu, S. M., Kim, S. L., Chung, Y. S., & Baek, J. H. (2025). Integration of smart sensors and IOT in precision agriculture: trends, challenges and future perspectives. In *Frontiers in Plant Science* (Vol. 16). Frontiers Media SA. <https://doi.org/10.3389/fpls.2025.1587869>
- Martelli, A., Rapinesi, D., Verdi, L., Donati, I. I. M., Dalla Marta, A., & Altobelli, F. (2025). Smart irrigation for management of processing tomato: a machine learning approach. *Irrigation Science*, 43(6), 1407–1424. <https://doi.org/10.1007/s00271-024-00993-9>
- Mohammed, B. (2025). A Comprehensive Overview of Federated Learning for Next-Generation Smart Agriculture: Current Trends, Challenges, and Future Directions. In *Informatica (Slovenia)* (Vol. 49, Number 1, pp. 117–136). Slovene Society Informatika. <https://doi.org/10.31449/inf.v49i1.6764>
- Nugrahn Halawa, D. (n.d.). *Peran Teknologi Pertanian Cerdas (Smart Farming) untuk Generasi Pertanian Indonesia* (Vol. 6).
- Omol, E., Mburu, L., Abuonji, P., & Onyango, D. (2024). Anomaly Detection In IoT Sensor Data Using Machine Learning Techniques For Predictive Maintenance In Smart Grids. In *International Journal of Science*. <http://ijstm.inarah.co.id>
- R. Vidhya, D. Lognathan, S. S., P.N. Periyasamy, & S. Sumathi. (2025). Anomaly Detection in IoT Networks Using Federated Machine Learning Approaches. *International Journal of Computational and Experimental Science and Engineering*, 11(3). <https://doi.org/10.22399/ijcesen.2485>
- Rhachi, H., Balboul, Y., & Bouayad, A. (2025). Enhanced Anomaly Detection in IoT Networks Using Deep Autoencoders with Feature Selection Techniques. *Sensors*, 25(10). <https://doi.org/10.3390/s25103150>

- Sivaprasad, S., & Fritz, M. (2024). *Don't Miss Out on Novelty: Importance of Novel Features for Deep Anomaly Detection*. <http://arxiv.org/abs/2310.00797>
- Sunarto, M. W., Kurniawan, D., Siswanto, E., & Huda, H. I. (2022). *Deteksi Anomali Menggunakan Extended Isolation Forest (Eif)*. 1(2), 96–111. <http://journal.stiestekom.ac.id/index.php/TEKNIK> page111
- Walczyna, T., Jankowski, D., & Piotrowski, Z. (2025). Enhancing Anomaly Detection Through Latent Space Manipulation in Autoencoders: A Comparative Analysis. *Applied Sciences (Switzerland)*, 15(1). <https://doi.org/10.3390/app15010286>
- Xiang, H., Zhang, X., Xu, X., Beheshti, A., Qi, L., Hong, Y., & Dou, W. (2024). Federated Learning-Based Anomaly Detection with Isolation Forest in the IoT-Edge Continuum. *ACM Transactions on Multimedia Computing, Communications, and Applications*. <https://doi.org/10.1145/3702995>